

Multi-Method Game Simulation Report

Generated: 2026-02-17 23:21 UTC

Games analyzed: 19

Algorithms: PPO, A2C, DQN

Seeds per game: 30 | Train steps: 280 | Eval steps: 140

Tell scales: [0.0, 0.5, 1.0, 1.5]

Summary: All Games Ranked by Best Mean Reward

Rank	Game	Best Algo	Mean	95% CI	p(vs 0)
1	iterated_prisoners_dilemm	PPO	+0.4540	[+0.2700, +0.6379]	1.32e-06***
2	hawk_dove	DQN	+0.3156	[+0.1661, +0.4651]	3.51e-05***
3	stag_hunt	PPO	+0.2959	[+0.1152, +0.4765]	0.0013**
4	blackjack	PPO	-0.1104	[-0.9642, +0.7433]	0.7998
5	signaling	DQN	-0.1508	[-0.2740, -0.0275]	0.0165*
6	crisis_negotiations	A2C	-0.1977	[-0.3368, -0.0585]	0.0054**
7	beer_distribution	DQN	-0.2242	[-0.3547, -0.0937]	7.58e-04***
8	principal_agent	DQN	-0.2486	[-0.3607, -0.1365]	1.38e-05***
9	market_for_lemons	DQN	-0.2581	[-0.3796, -0.1366]	3.13e-05***
10	poker	A2C	-0.2820	[-0.4079, -0.1561]	1.13e-05***
11	chess	DQN	-0.3123	[-0.3958, -0.2288]	2.28e-13***
12	trust_game	A2C	-0.3141	[-0.4332, -0.1951]	2.33e-07***
13	auctions	PPO	-0.3370	[-0.4436, -0.2305]	5.70e-10***
14	cournot	A2C	-0.3401	[-0.4427, -0.2375]	8.30e-11***
15	interrogation	DQN	-0.3478	[-0.4527, -0.2429]	8.08e-11***
16	ultimatum	A2C	-0.3515	[-0.4708, -0.2321]	7.79e-09***
17	tenders	DQN	-0.3586	[-0.4254, -0.2919]	< 1e-16***
18	bertrand	DQN	-0.3741	[-0.4767, -0.2714]	9.06e-13***
19	public_goods	PPO	-0.3819	[-0.5071, -0.2566]	2.29e-09***

Game: auctions
difficulty=0.67 tell_weight=0.4 profile_weight=0.35

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3620	0.2516	0.0459	[-0.4521, -0.2720]	3.11e-15***	30
DQN	-0.3673	0.2851	0.0521	[-0.4693, -0.2652]	1.72e-12***	30
PPO	-0.3370	0.2978	0.0544	[-0.4436, -0.2305]	5.70e-10***	30

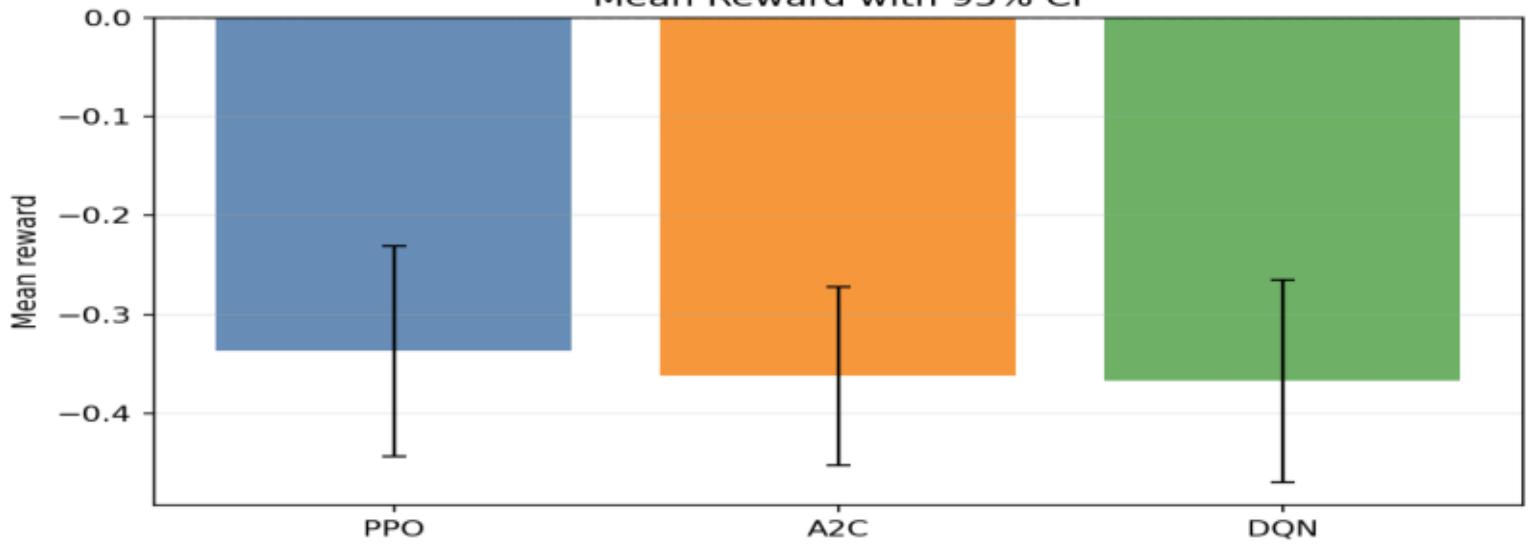
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0052	[-0.0370, +0.0475]	0.8080
PPO - A2C	+0.0250	[-0.0197, +0.0696]	0.2726
PPO - DQN	+0.0302	[+0.0035, +0.0569]	0.0264*

auctions - Charts

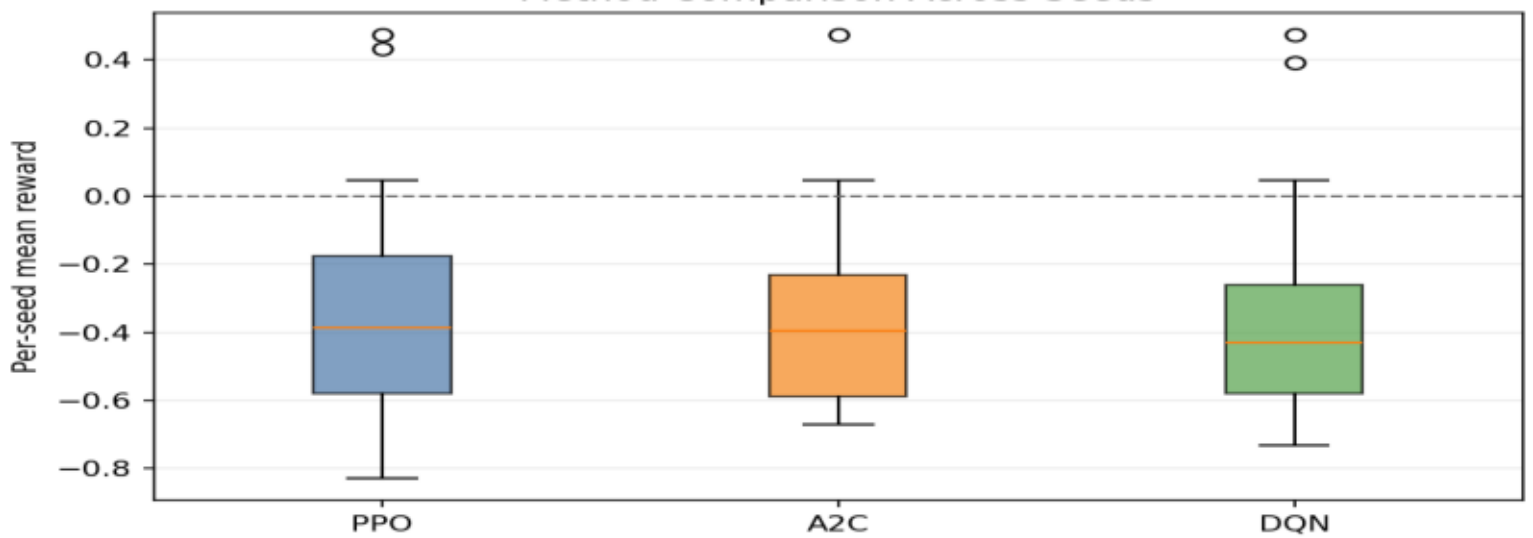
Mean Reward with 95% CI

Mean Reward with 95% CI



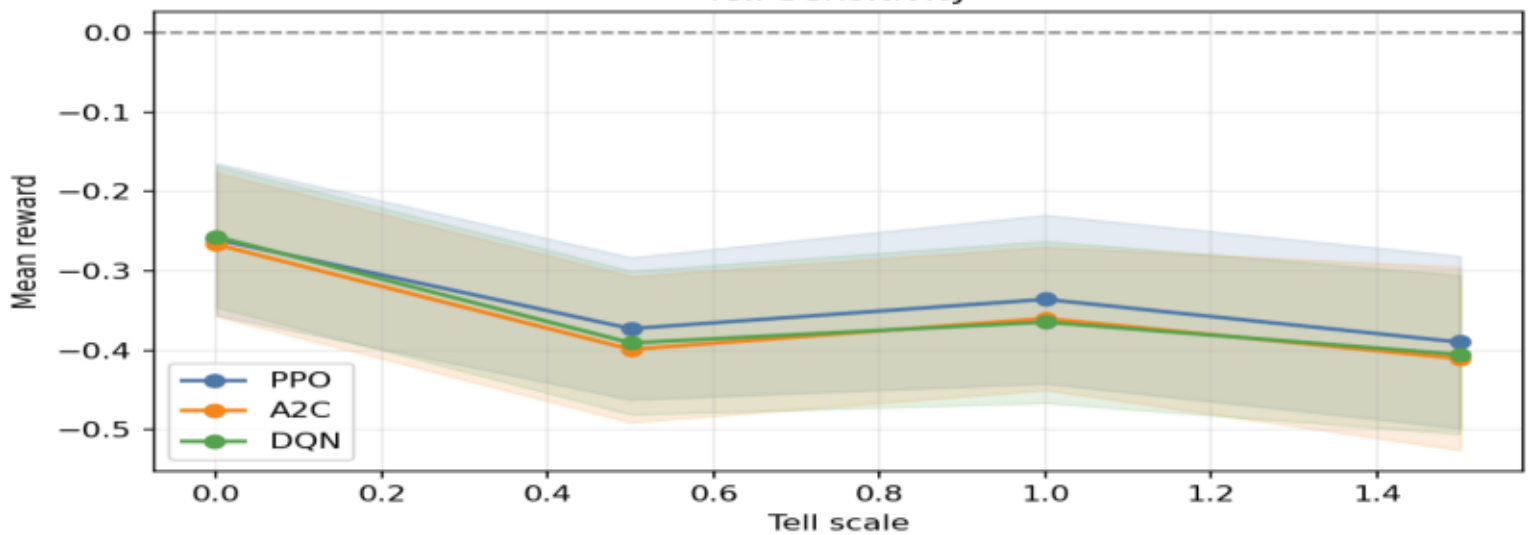
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



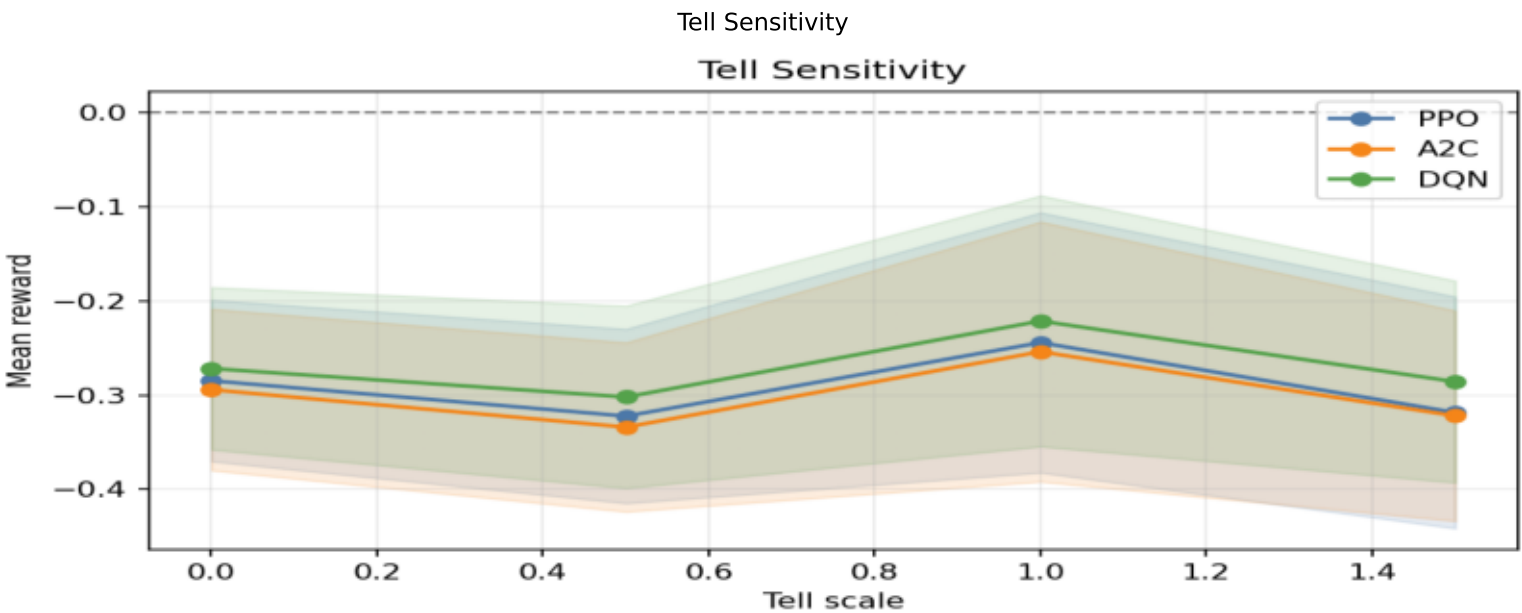
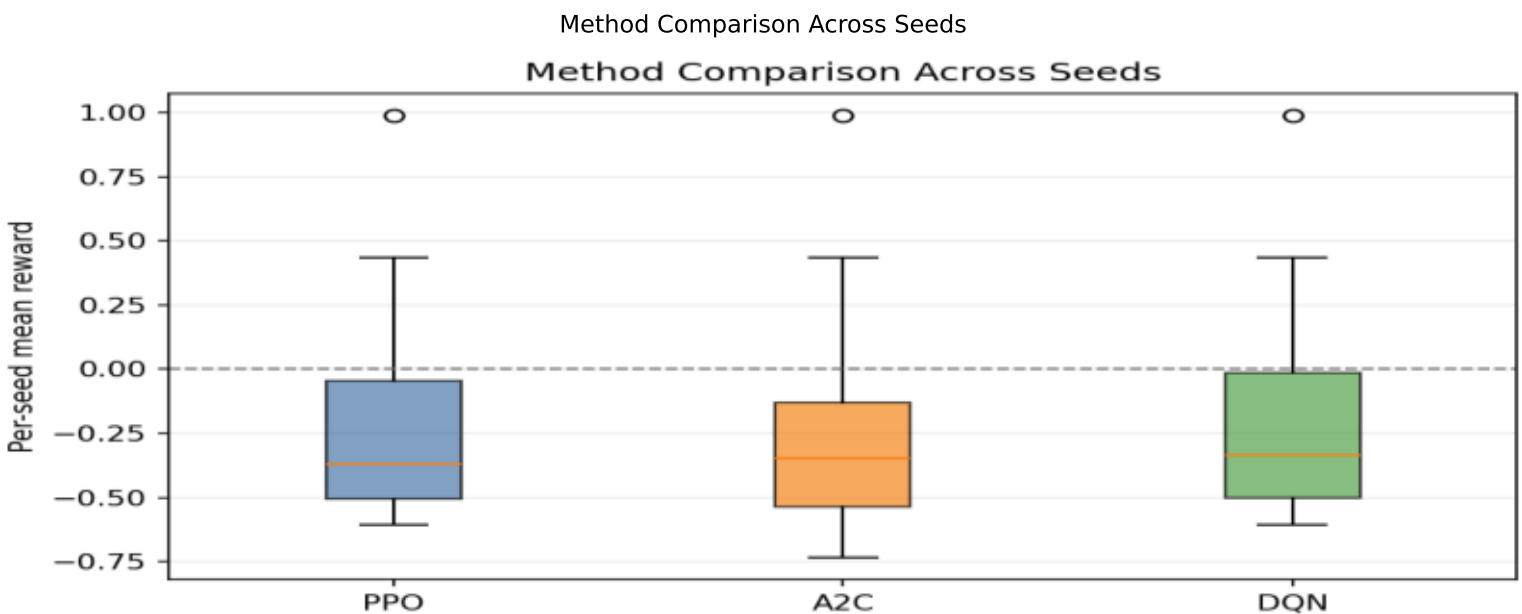
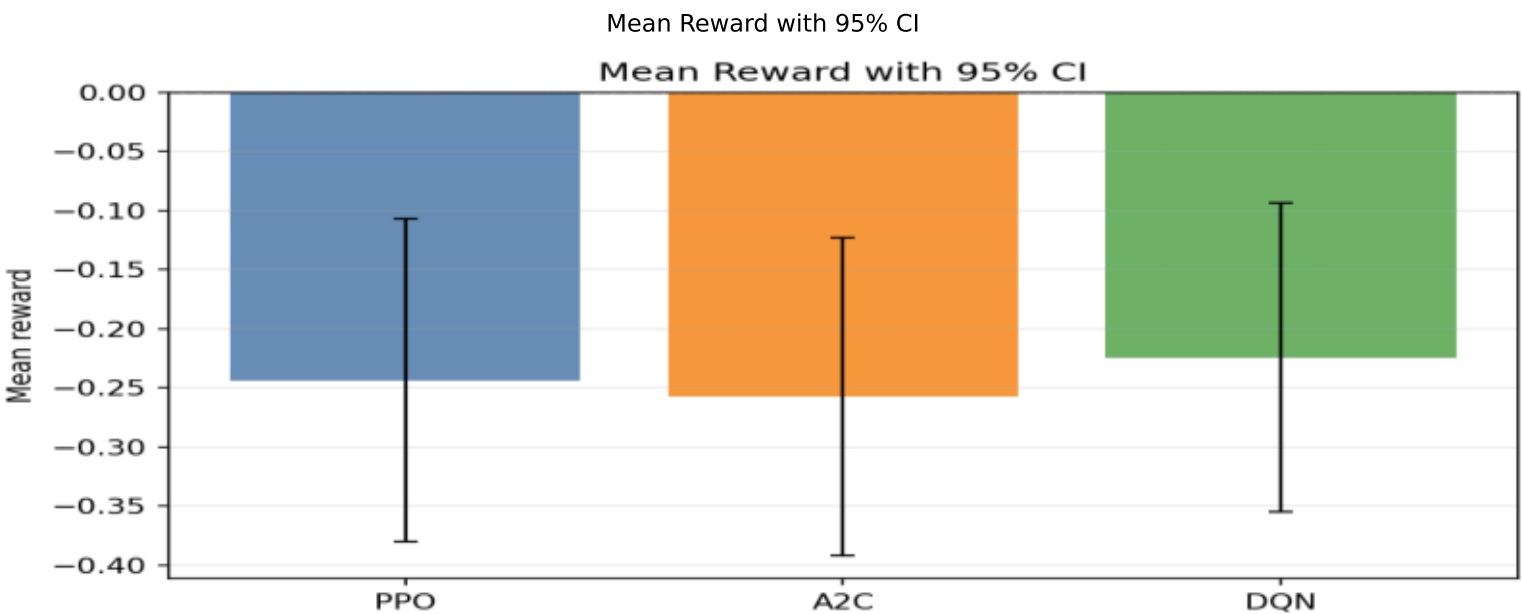
Game: beer_distribution
difficulty=0.66 tell_weight=0.29 profile_weight=0.3

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.2572	0.3757	0.0686	[-0.3917, -0.1228]	1.77e-04***	30
DQN	-0.2242	0.3646	0.0666	[-0.3547, -0.0937]	7.58e-04***	30
PPO	-0.2436	0.3810	0.0696	[-0.3799, -0.1072]	4.62e-04***	30

Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0331	[-0.0887, +0.0226]	0.2442
PPO - A2C	+0.0137	[-0.0505, +0.0779]	0.6764
PPO - DQN	-0.0194	[-0.0497, +0.0109]	0.2100

beer_distribution - Charts



Game: bertrand
difficulty=0.64 tell_weight=0.31 profile_weight=0.33

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3858	0.2827	0.0516	[-0.4869, -0.2847]	7.66e-14***	30
DQN	-0.3741	0.2868	0.0524	[-0.4767, -0.2714]	9.06e-13***	30
PPO	-0.3820	0.2942	0.0537	[-0.4873, -0.2767]	1.14e-12***	30

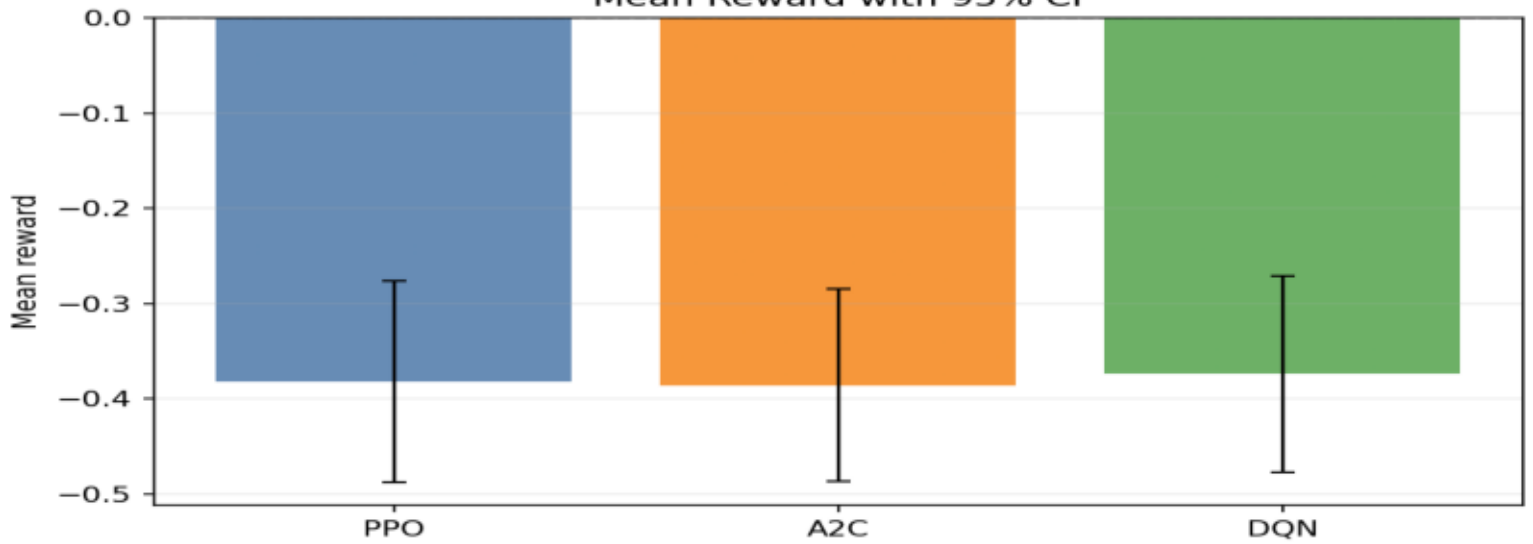
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0117	[-0.0698, +0.0464]	0.6925
PPO - A2C	+0.0038	[-0.0343, +0.0419]	0.8447
PPO - DQN	-0.0079	[-0.0724, +0.0566]	0.8101

bertrand - Charts

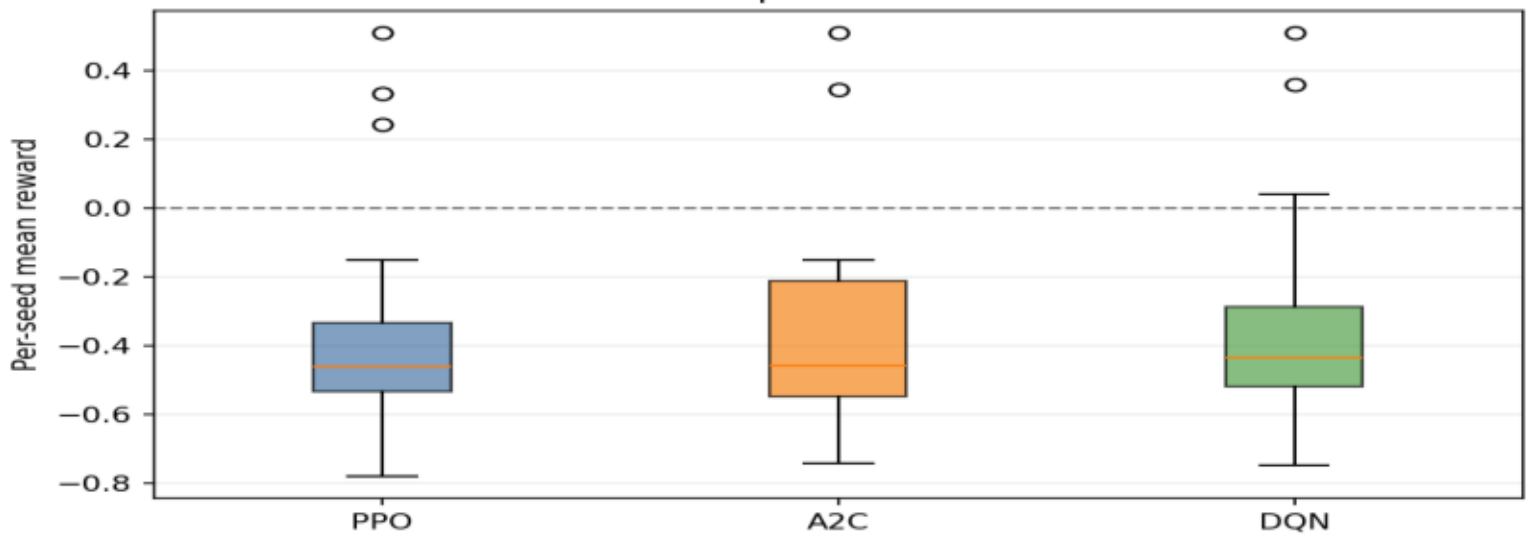
Mean Reward with 95% CI

Mean Reward with 95% CI



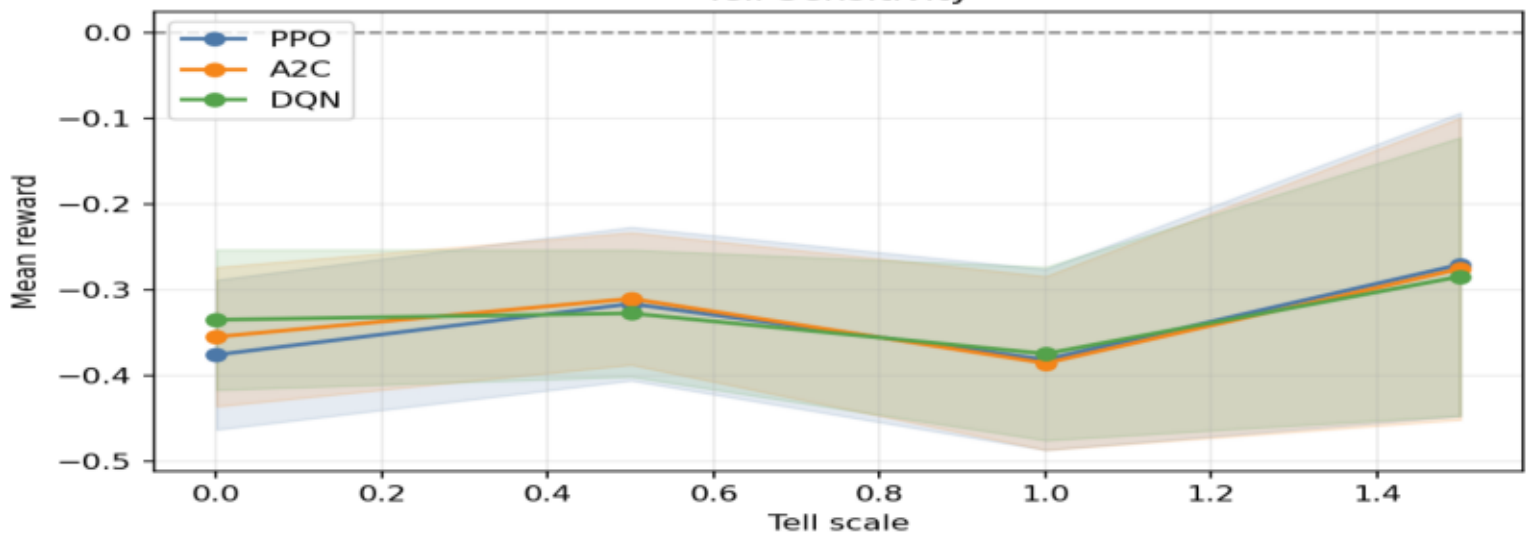
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: blackjack
difficulty=0.56 tell_weight=0.34 profile_weight=0.28

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.1224	0.7462	0.4308	[-0.9668, +0.7220]	0.7763	3
DQN	-0.2222	0.6587	0.3803	[-0.9676, +0.5231]	0.5590	3
PPO	-0.1104	0.7544	0.4356	[-0.9642, +0.7433]	0.7998	3

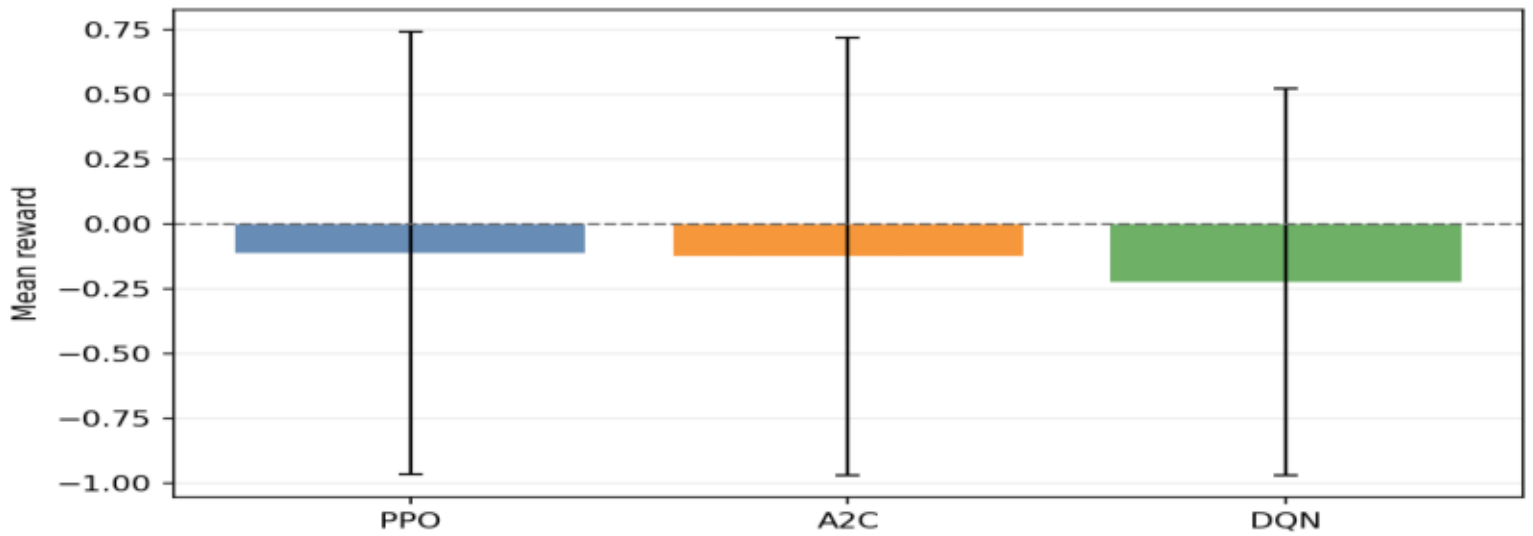
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0998	[-0.1240, +0.3236]	0.3820
PPO - A2C	+0.0120	[-0.0115, +0.0354]	0.3173
PPO - DQN	+0.1118	[-0.0998, +0.3233]	0.3004

blackjack - Charts

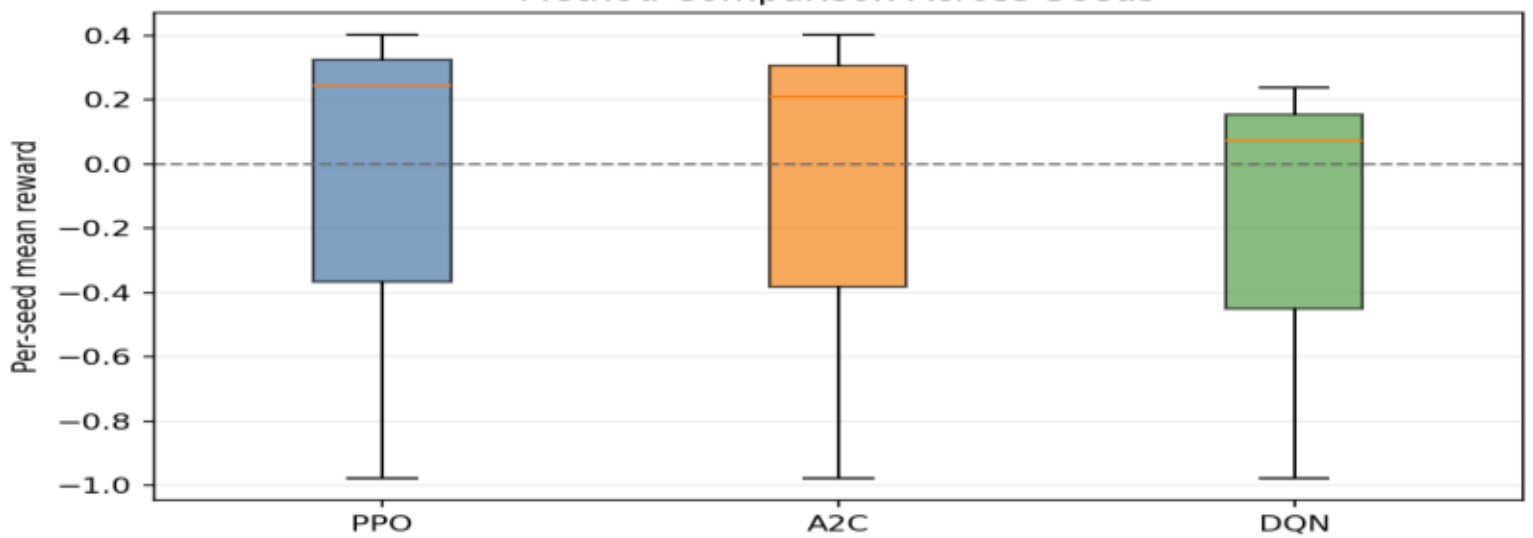
Mean Reward with 95% CI

Mean Reward with 95% CI



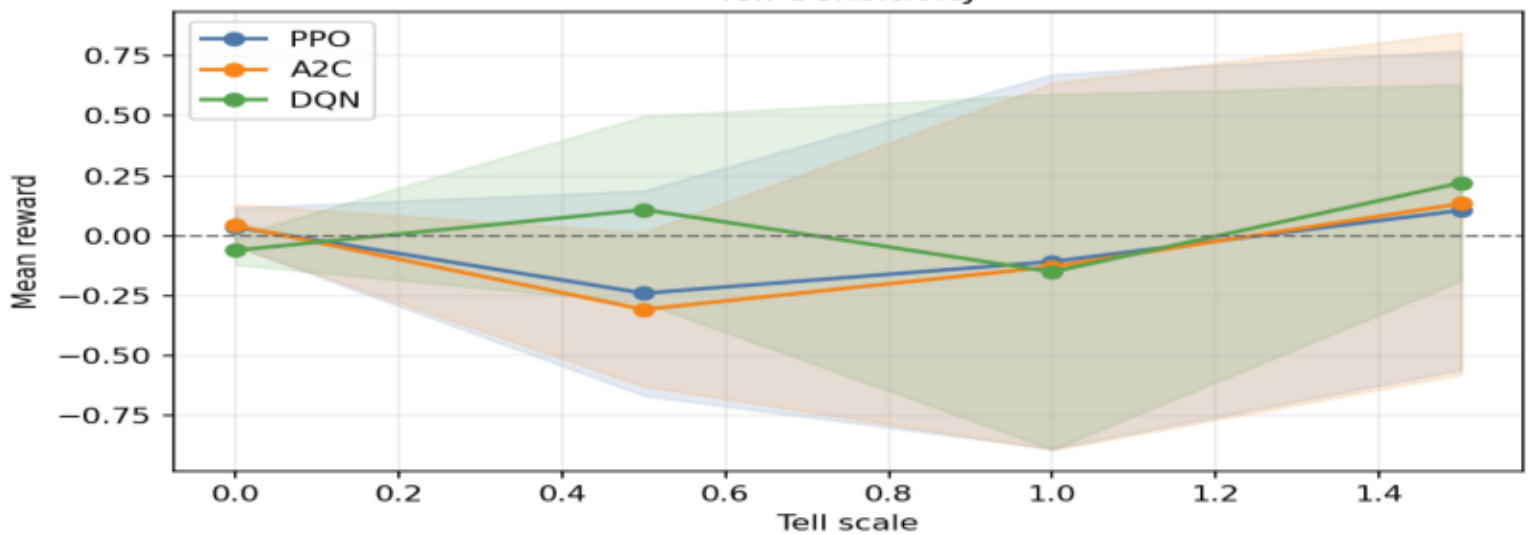
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: chess
difficulty=0.72 tell_weight=0.38 profile_weight=0.26

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3255	0.2437	0.0445	[-0.4127, -0.2383]	2.54e-13***	30
DQN	-0.3123	0.2333	0.0426	[-0.3958, -0.2288]	2.28e-13***	30
PPO	-0.3440	0.2358	0.0431	[-0.4284, -0.2596]	1.33e-15***	30

Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0132	[-0.0396, +0.0132]	0.3279
PPO - A2C	-0.0185	[-0.0484, +0.0114]	0.2259
PPO - DQN	-0.0317	[-0.0632, -0.0002]	0.0486*

chess - Charts

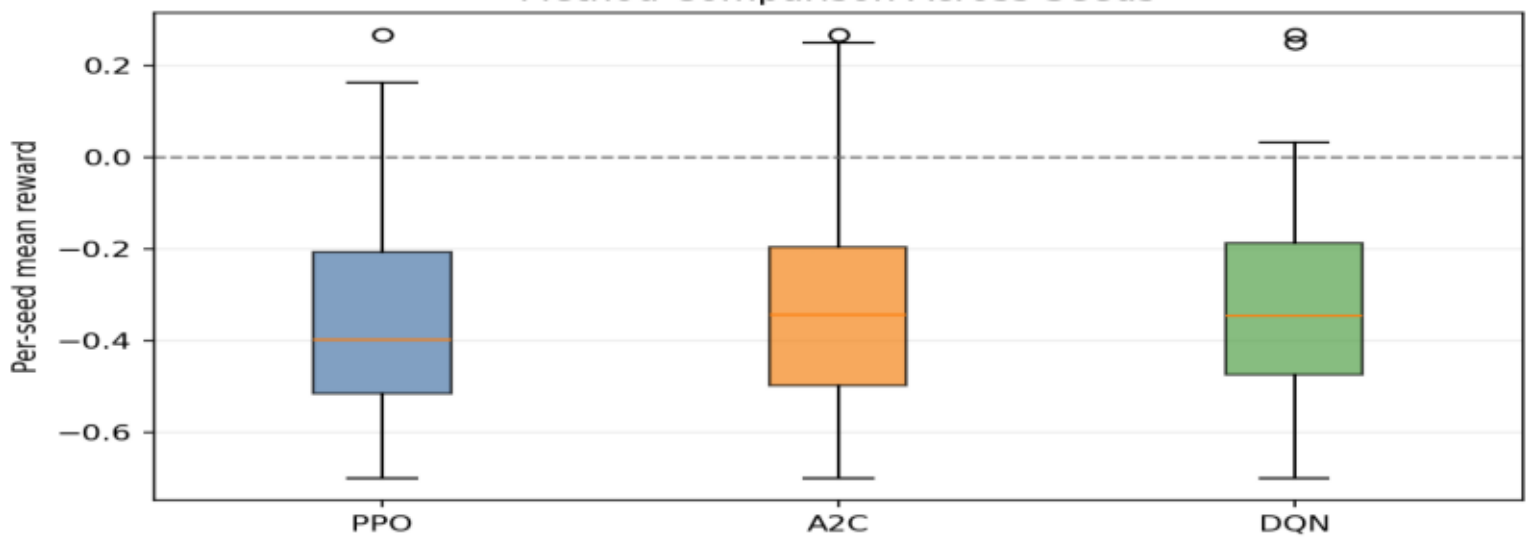
Mean Reward with 95% CI

Mean Reward with 95% CI



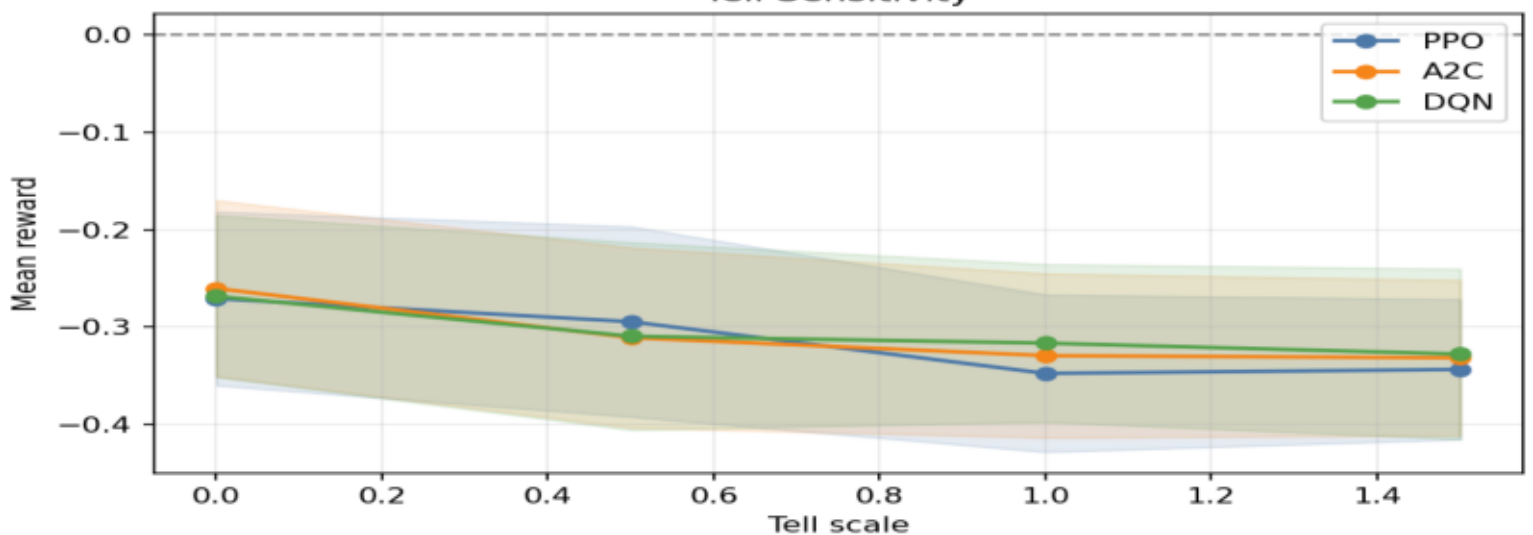
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



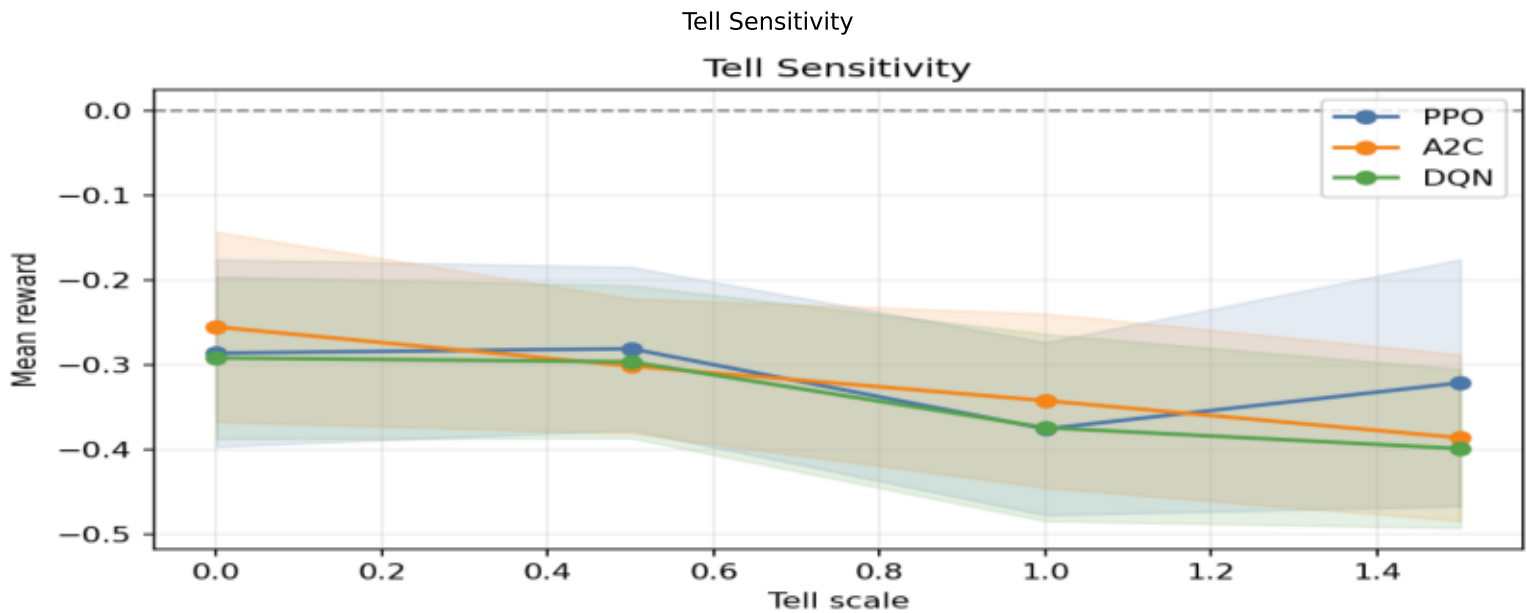
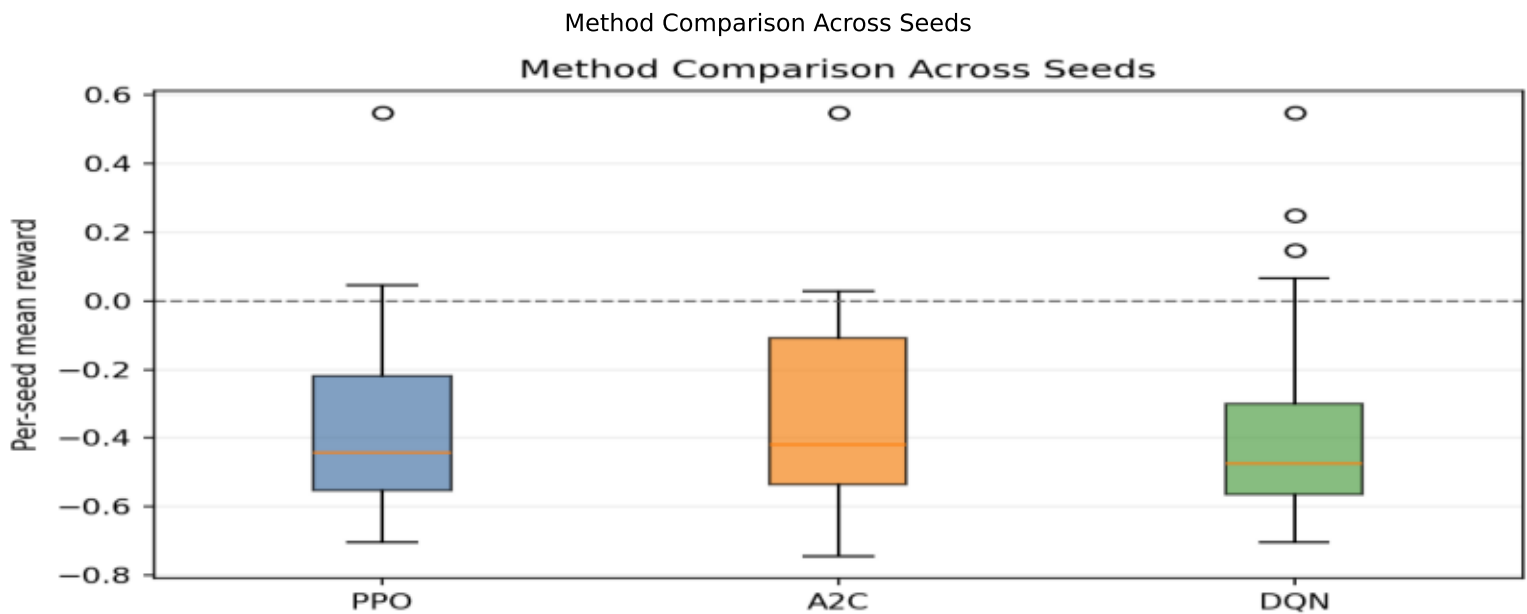
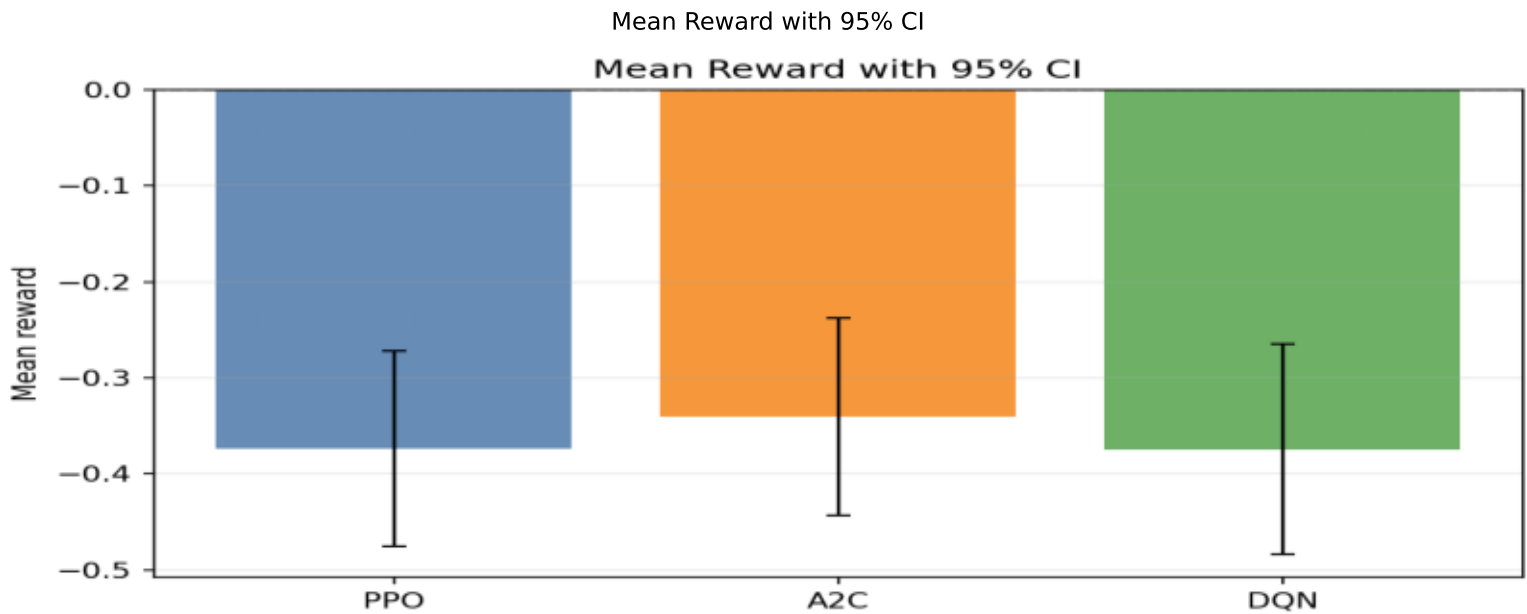
Game: cournot
difficulty=0.63 tell_weight=0.3 profile_weight=0.33

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3401	0.2868	0.0524	[-0.4427, -0.2375]	8.30e-11***	30
DQN	-0.3741	0.3048	0.0557	[-0.4832, -0.2650]	1.80e-11***	30
PPO	-0.3737	0.2842	0.0519	[-0.4754, -0.2720]	5.92e-13***	30

Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0340	[-0.0420, +0.1099]	0.3804
PPO - A2C	-0.0336	[-0.0968, +0.0296]	0.2967
PPO - DQN	+0.0004	[-0.0704, +0.0711]	0.9922

cournot - Charts



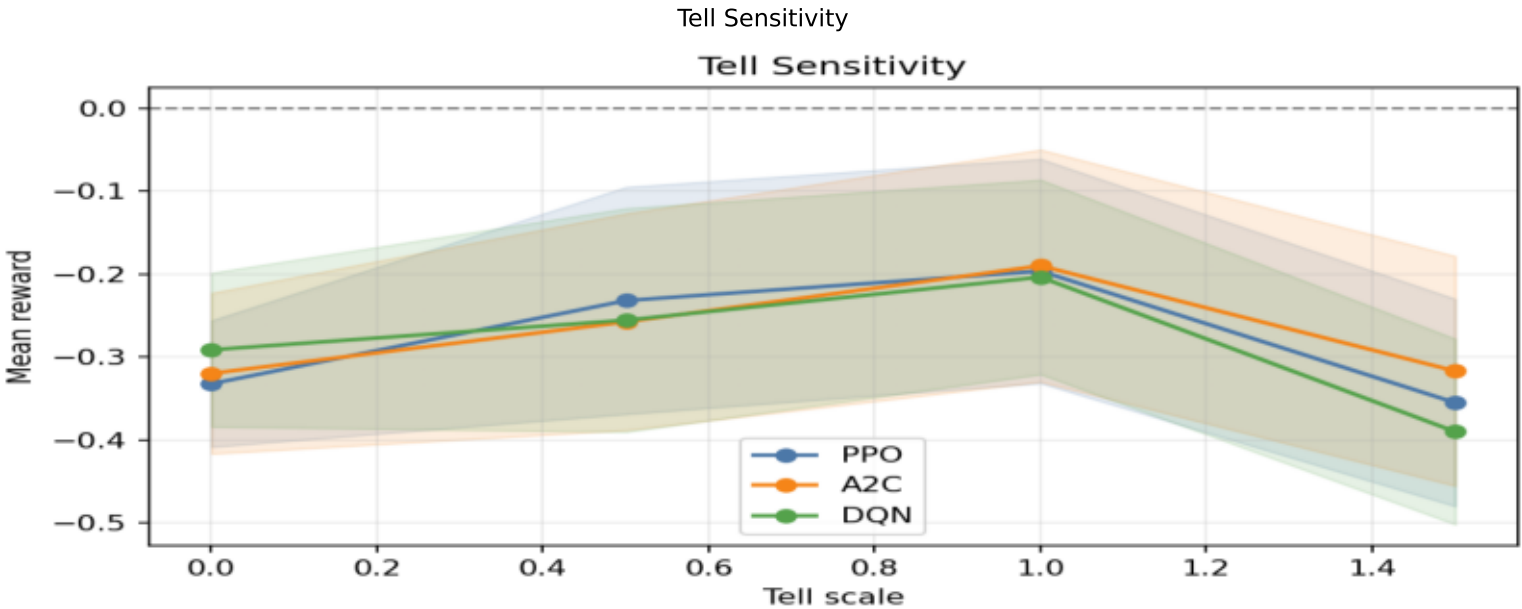
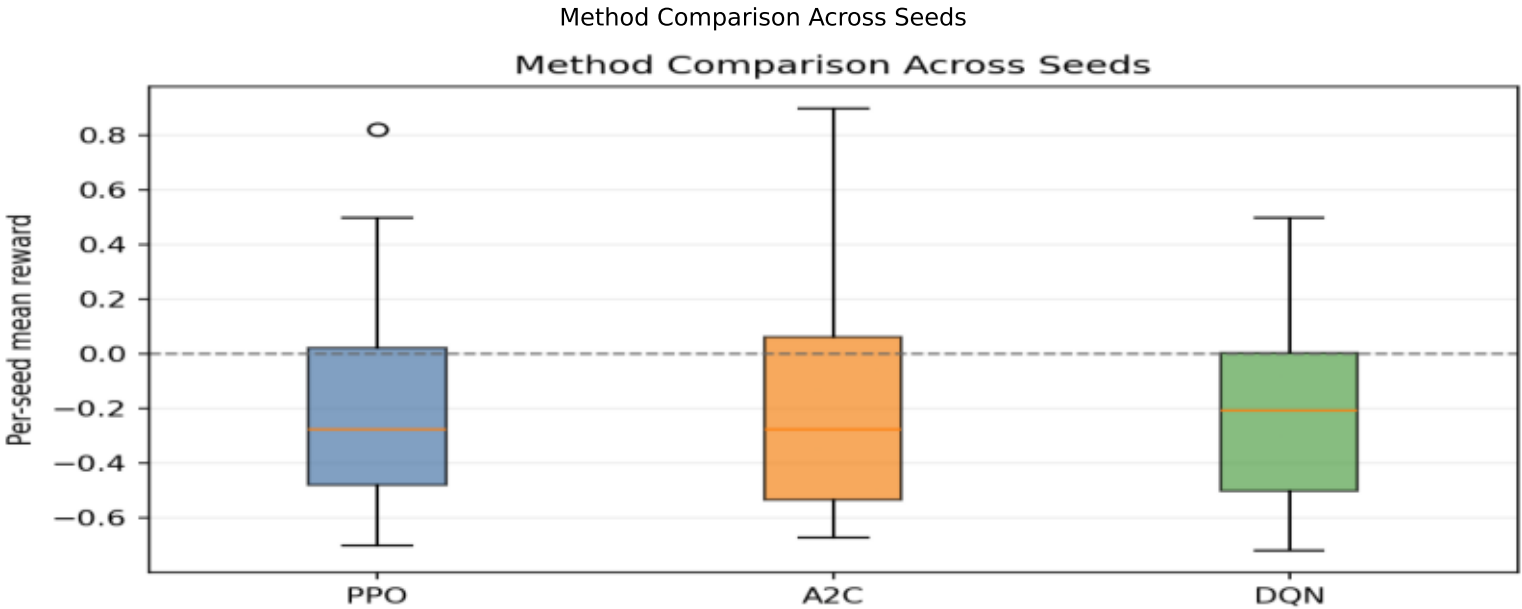
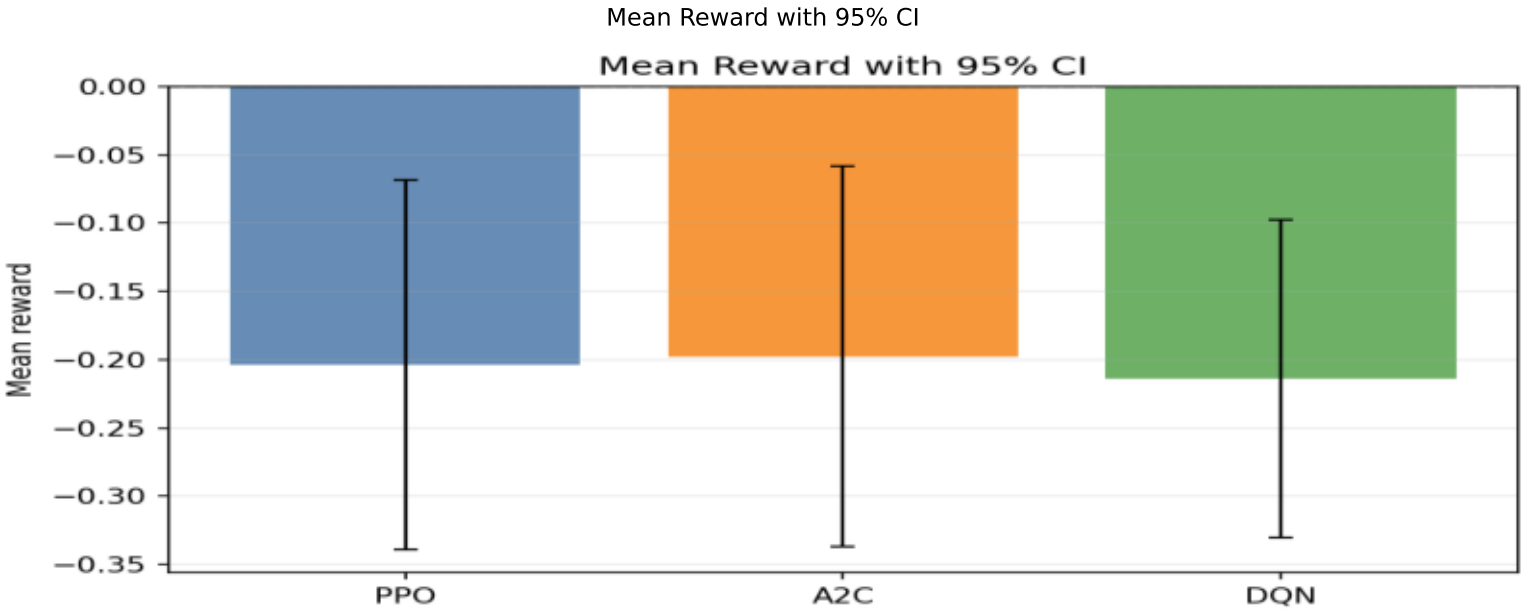
Game: crisis_negotiations
difficulty=0.73 tell_weight=0.46 profile_weight=0.34

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.1977	0.3889	0.0710	[-0.3368, -0.0585]	0.0054**	30
DQN	-0.2138	0.3253	0.0594	[-0.3302, -0.0974]	3.19e-04***	30
PPO	-0.2035	0.3778	0.0690	[-0.3387, -0.0683]	0.0032**	30

Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0161	[-0.0677, +0.1000]	0.7060
PPO - A2C	-0.0058	[-0.0659, +0.0542]	0.8485
PPO - DQN	+0.0103	[-0.0849, +0.1055]	0.8323

crisis_negotiations - Charts



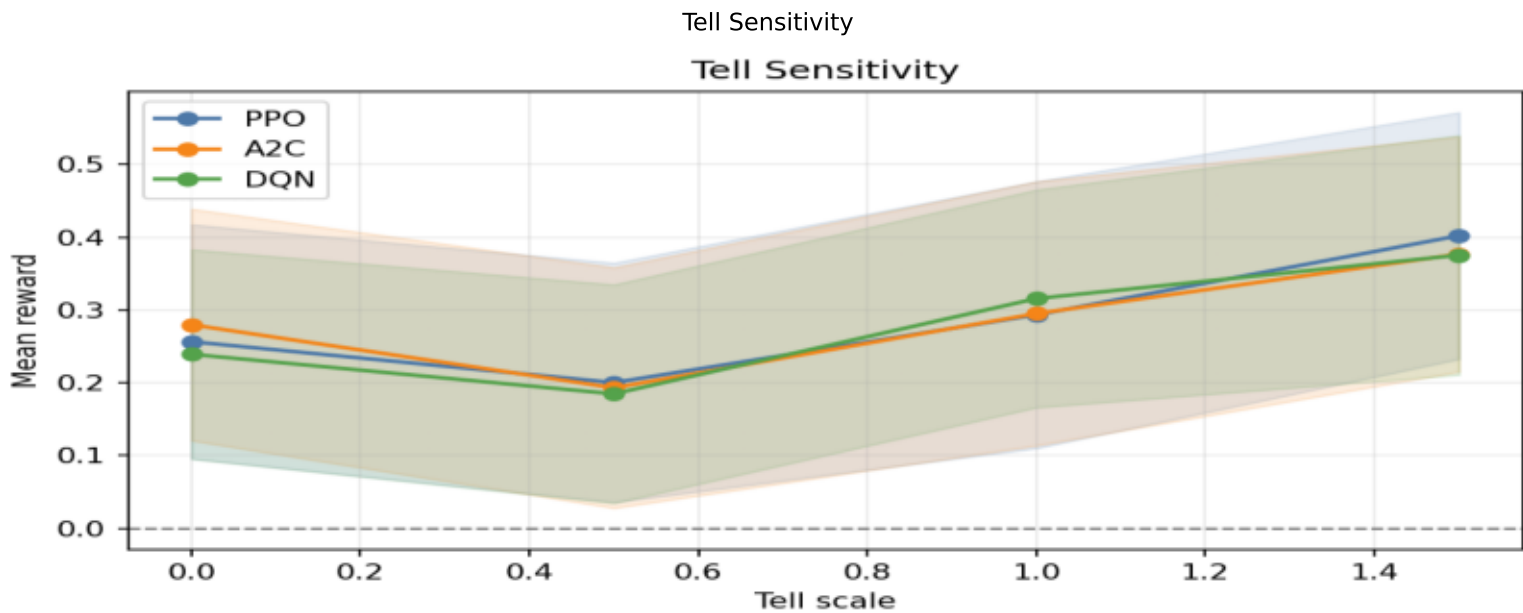
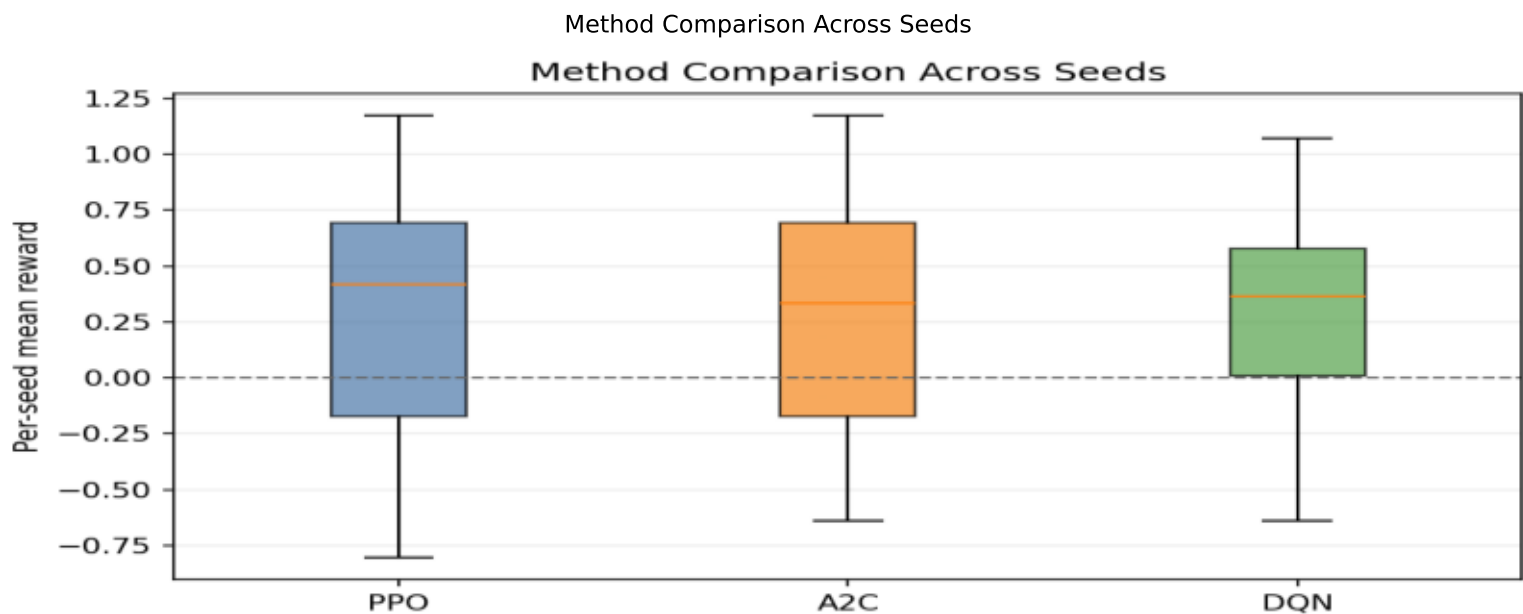
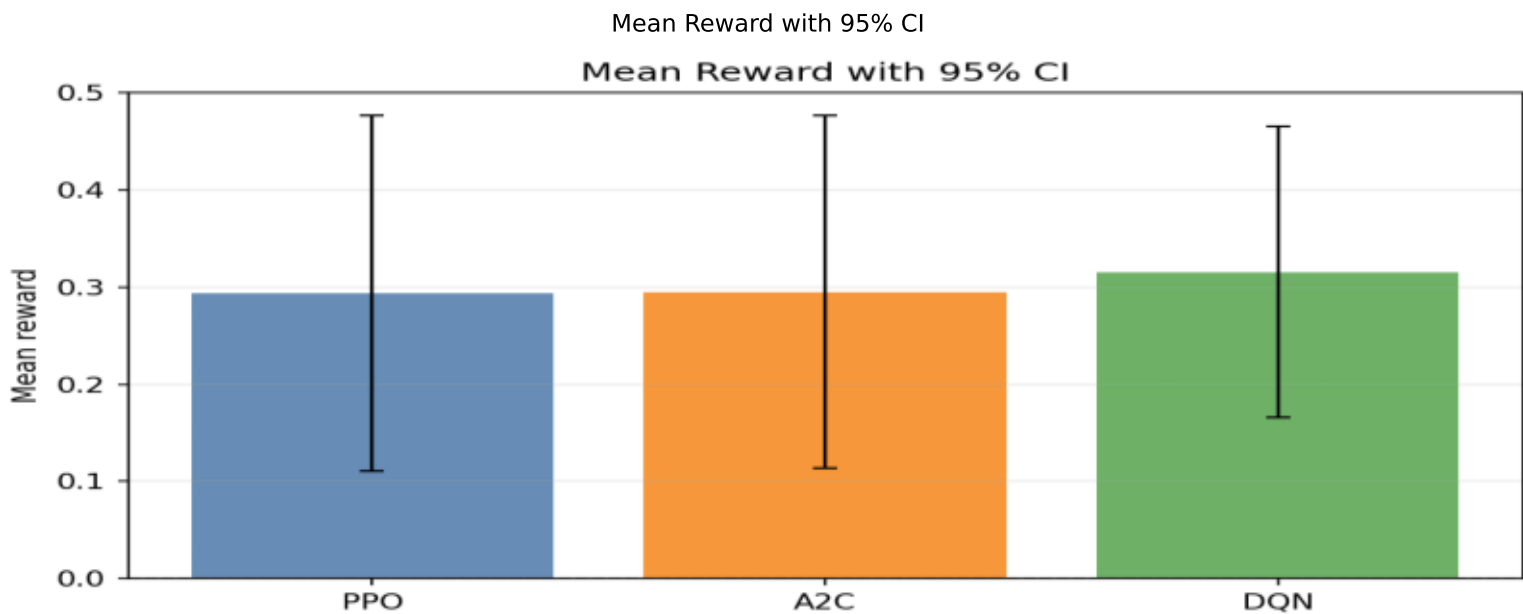
Game: hawk_dove
difficulty=0.57 tell_weight=0.35 profile_weight=0.32

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	+0.2951	0.5068	0.0925	[+0.1137, +0.4765]	0.0014**	30
DQN	+0.3156	0.4178	0.0763	[+0.1661, +0.4651]	3.51e-05***	30
PPO	+0.2933	0.5122	0.0935	[+0.1100, +0.4766]	0.0017**	30

Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0205	[-0.1101, +0.0690]	0.6528
PPO - A2C	-0.0018	[-0.0230, +0.0194]	0.8697
PPO - DQN	-0.0223	[-0.1055, +0.0609]	0.5992

hawk_dove - Charts



Game: interrogation
difficulty=0.71 tell_weight=0.48 profile_weight=0.31

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3540	0.3298	0.0602	[-0.4720, -0.2360]	4.14e-09***	30
DQN	-0.3478	0.2931	0.0535	[-0.4527, -0.2429]	8.08e-11***	30
PPO	-0.3882	0.2984	0.0545	[-0.4950, -0.2814]	1.04e-12***	30

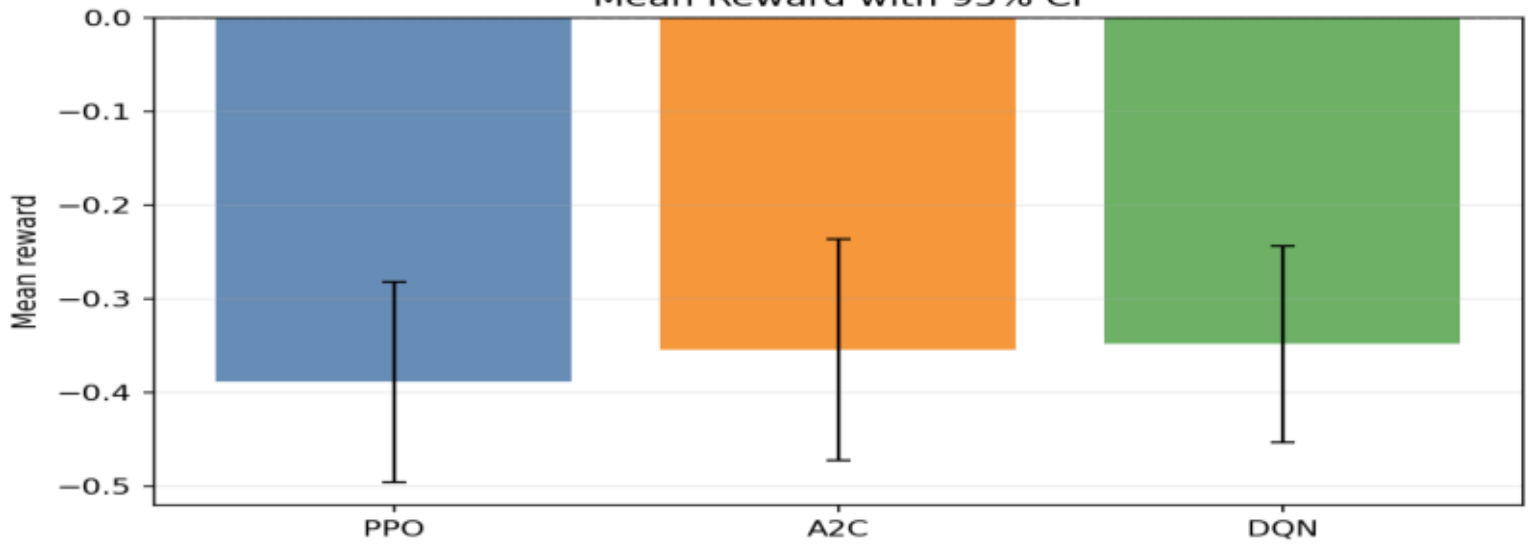
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0061	[-0.0576, +0.0453]	0.8149
PPO - A2C	-0.0343	[-0.1062, +0.0377]	0.3508
PPO - DQN	-0.0404	[-0.0928, +0.0120]	0.1309

interrogation - Charts

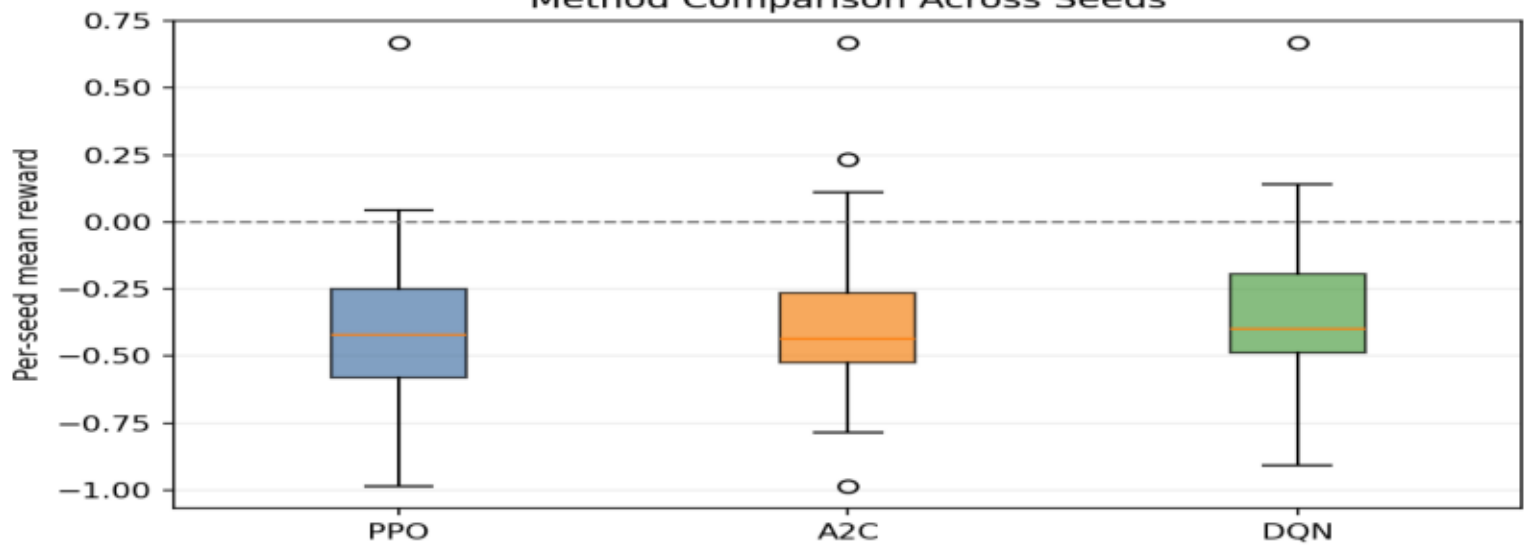
Mean Reward with 95% CI

Mean Reward with 95% CI



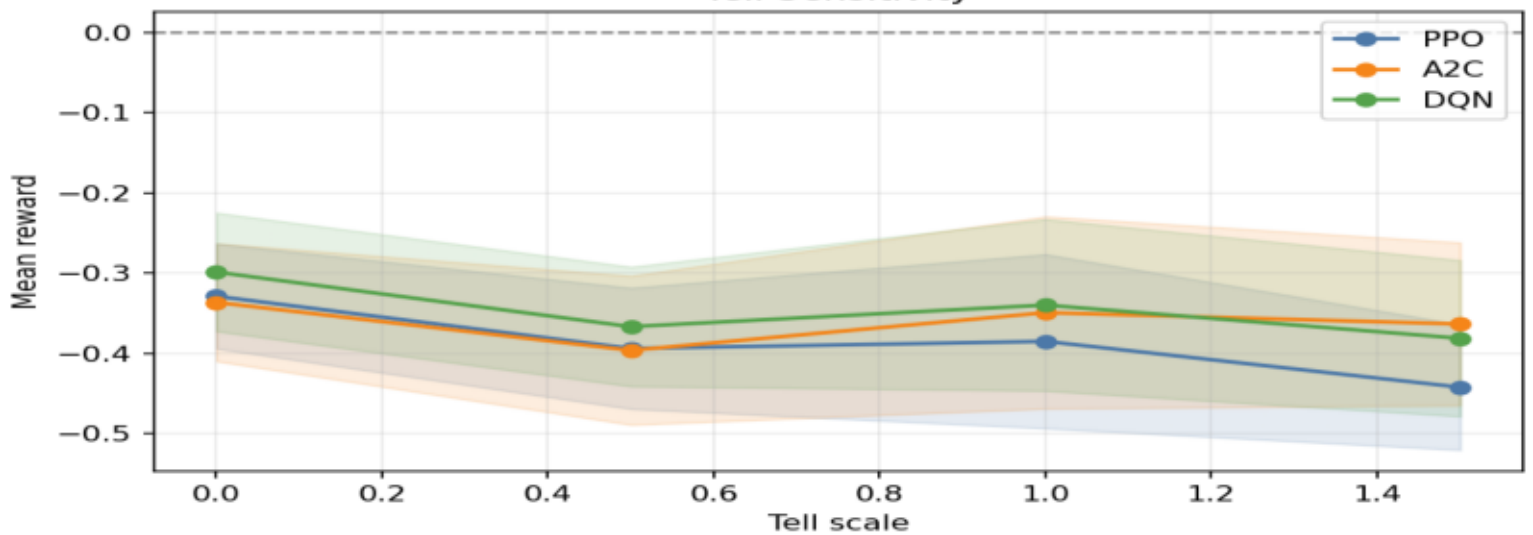
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: iterated_prisoners_dilemma
difficulty=0.56 tell_weight=0.34 profile_weight=0.31

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	+0.4522	0.5095	0.0930	[+0.2699, +0.6345]	1.16e-06***	30
DQN	+0.4471	0.4926	0.0899	[+0.2708, +0.6233]	6.67e-07***	30
PPO	+0.4540	0.5140	0.0939	[+0.2700, +0.6379]	1.32e-06***	30

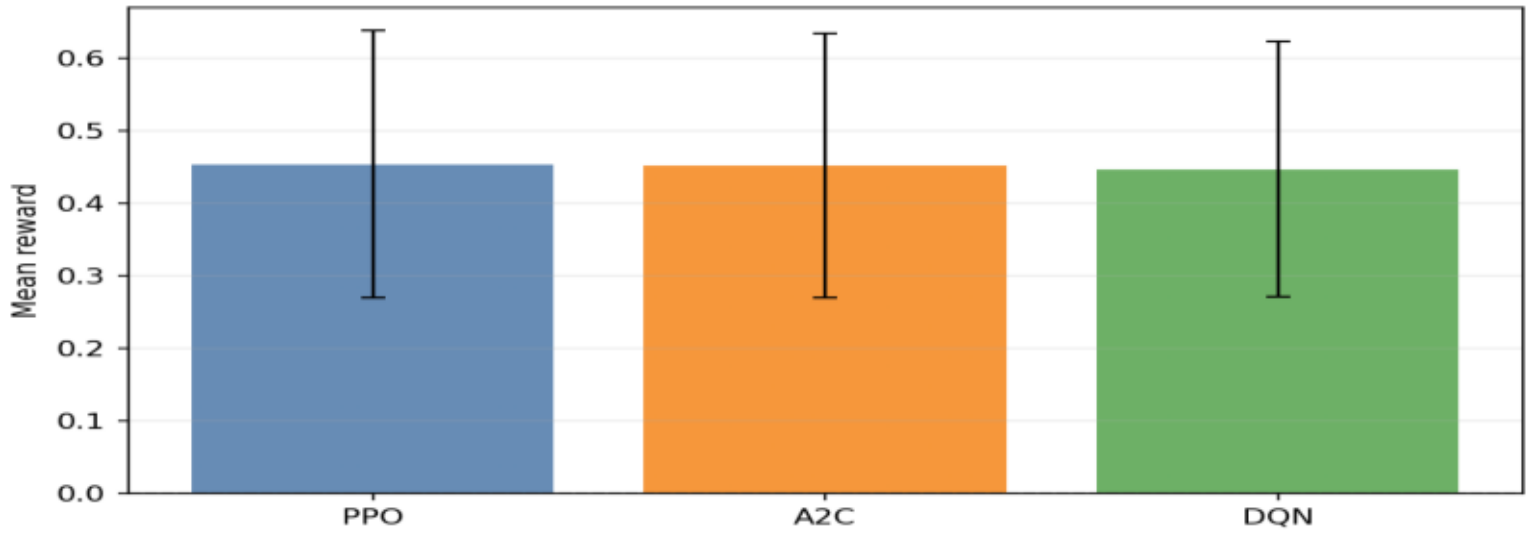
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0052	[-0.0229, +0.0332]	0.7175
PPO - A2C	+0.0017	[-0.0169, +0.0203]	0.8558
PPO - DQN	+0.0069	[-0.0272, +0.0410]	0.6914

iterated_prisoners_dilemma - Charts

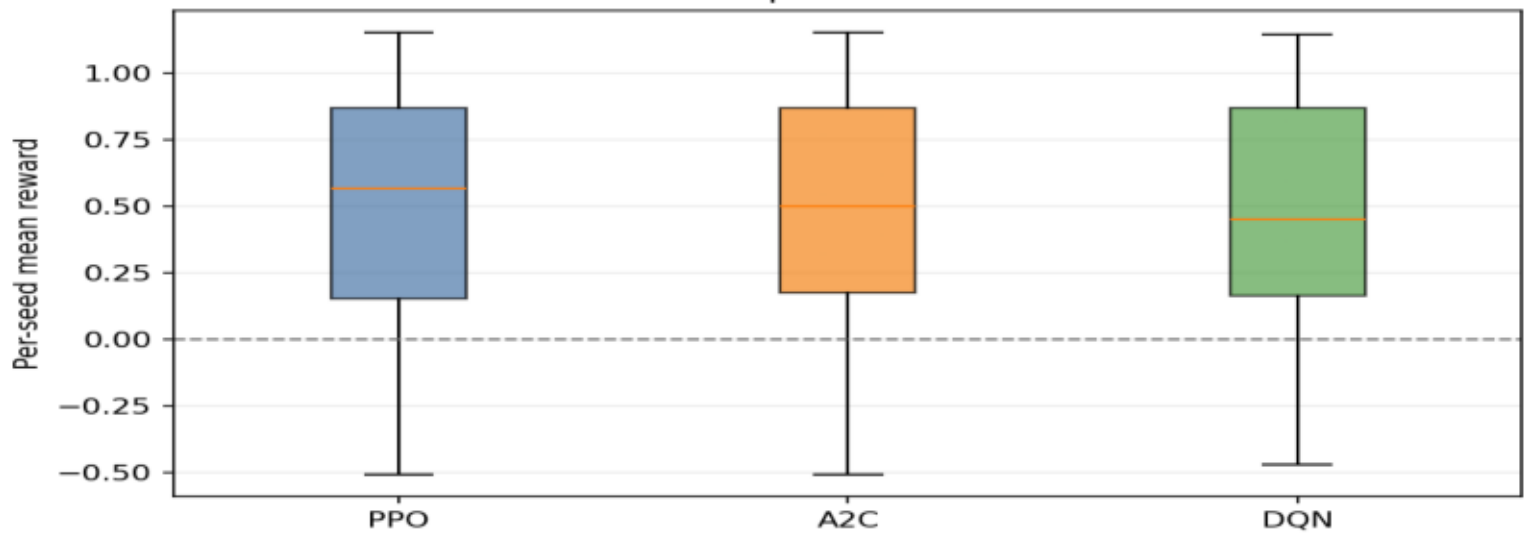
Mean Reward with 95% CI

Mean Reward with 95% CI



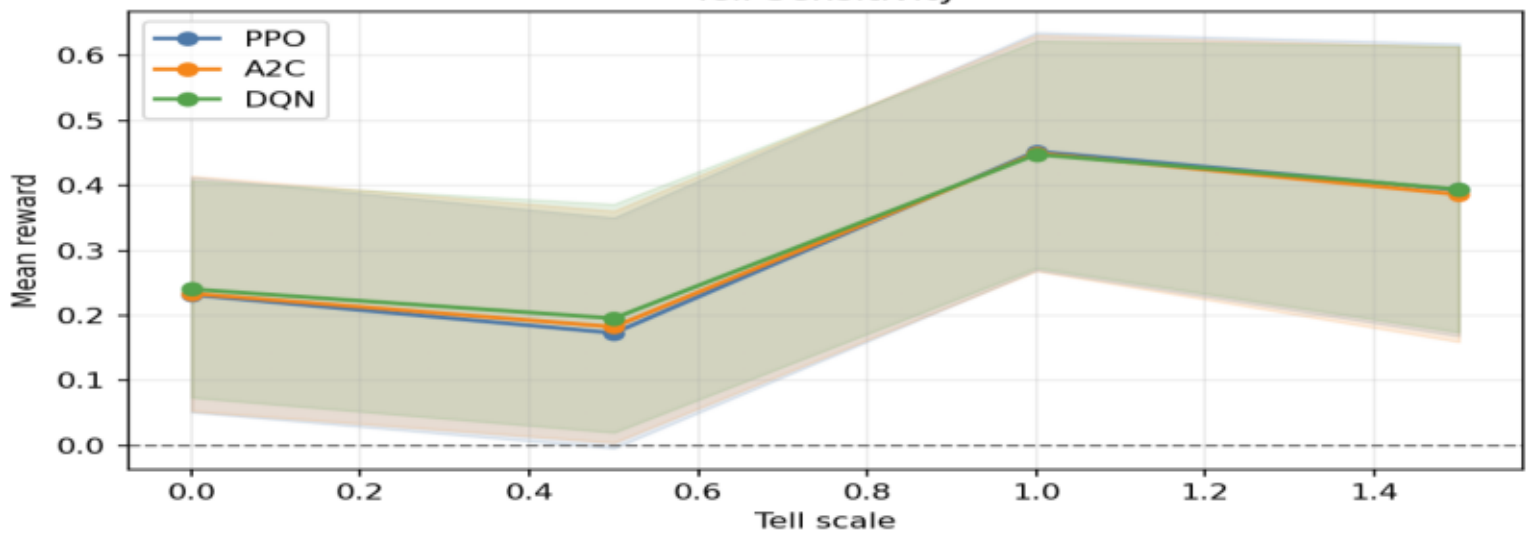
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



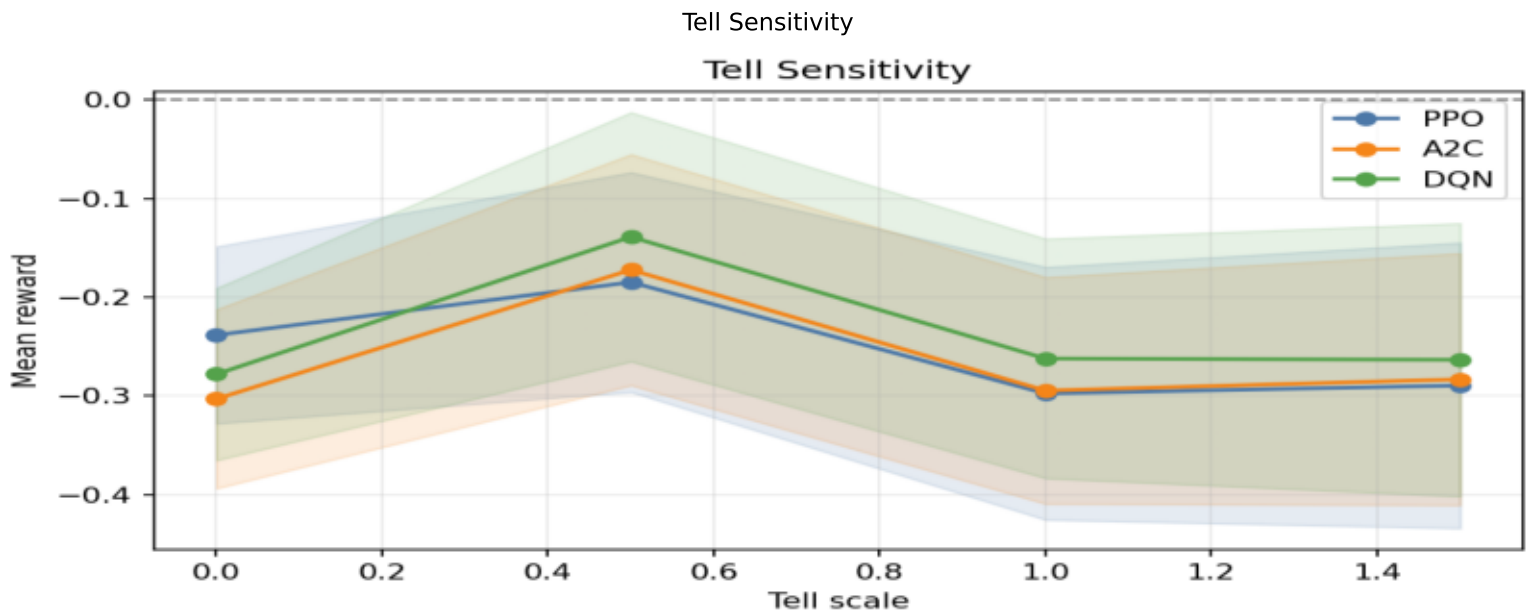
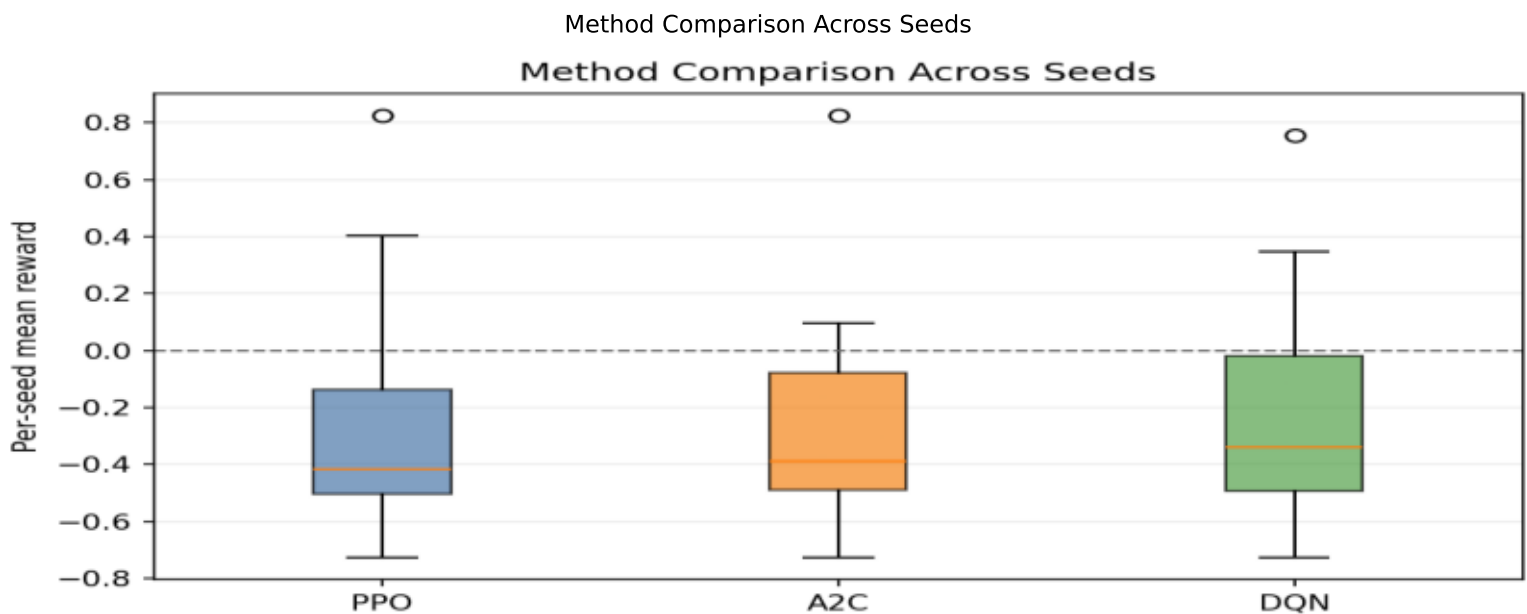
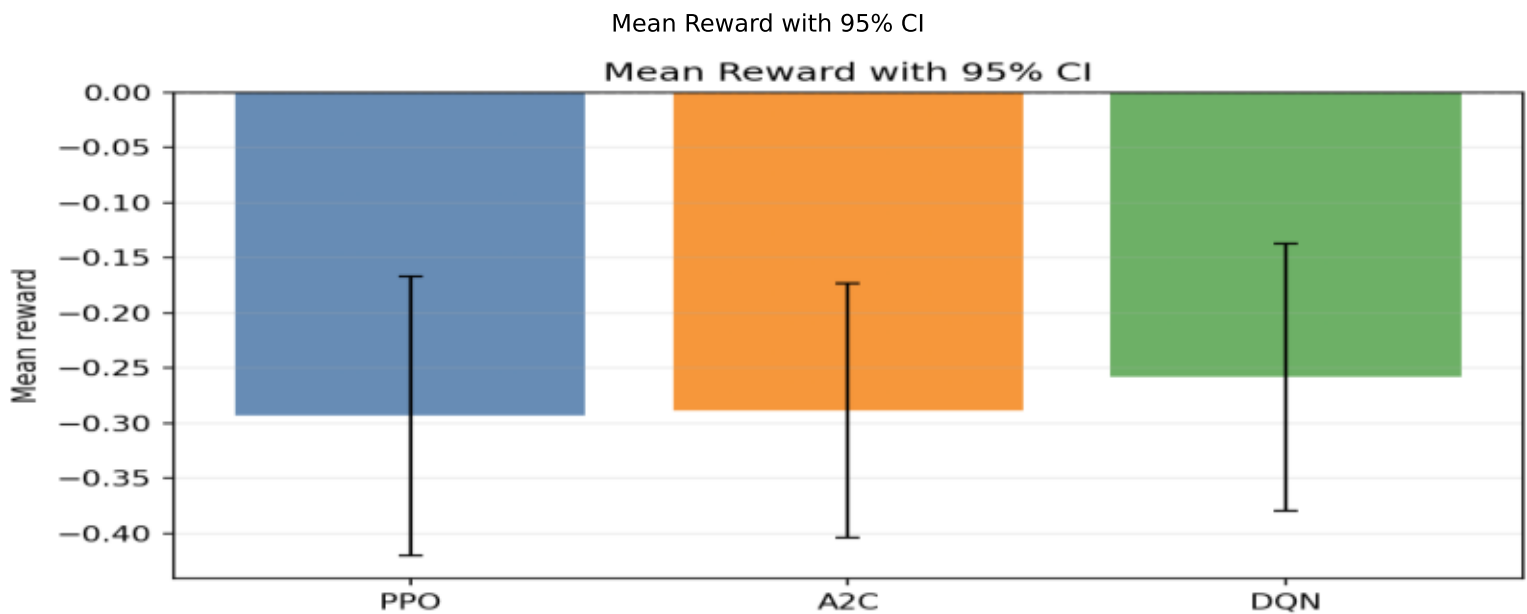
Game: market_for_lemons
difficulty=0.68 tell_weight=0.37 profile_weight=0.34

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.2886	0.3231	0.0590	[-0.4042, -0.1730]	1.00e-06***	30
DQN	-0.2581	0.3395	0.0620	[-0.3796, -0.1366]	3.13e-05***	30
PPO	-0.2931	0.3540	0.0646	[-0.4198, -0.1664]	5.75e-06***	30

Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0305	[-0.0909, +0.0300]	0.3237
PPO - A2C	-0.0045	[-0.0683, +0.0592]	0.8893
PPO - DQN	-0.0350	[-0.0911, +0.0211]	0.2215

market_for_lemons - Charts



Game: poker
difficulty=0.75 tell_weight=0.45 profile_weight=0.32

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.2820	0.3519	0.0642	[-0.4079, -0.1561]	1.13e-05***	30
DQN	-0.2851	0.3569	0.0652	[-0.4128, -0.1574]	1.21e-05***	30
PPO	-0.3071	0.3545	0.0647	[-0.4340, -0.1803]	2.08e-06***	30

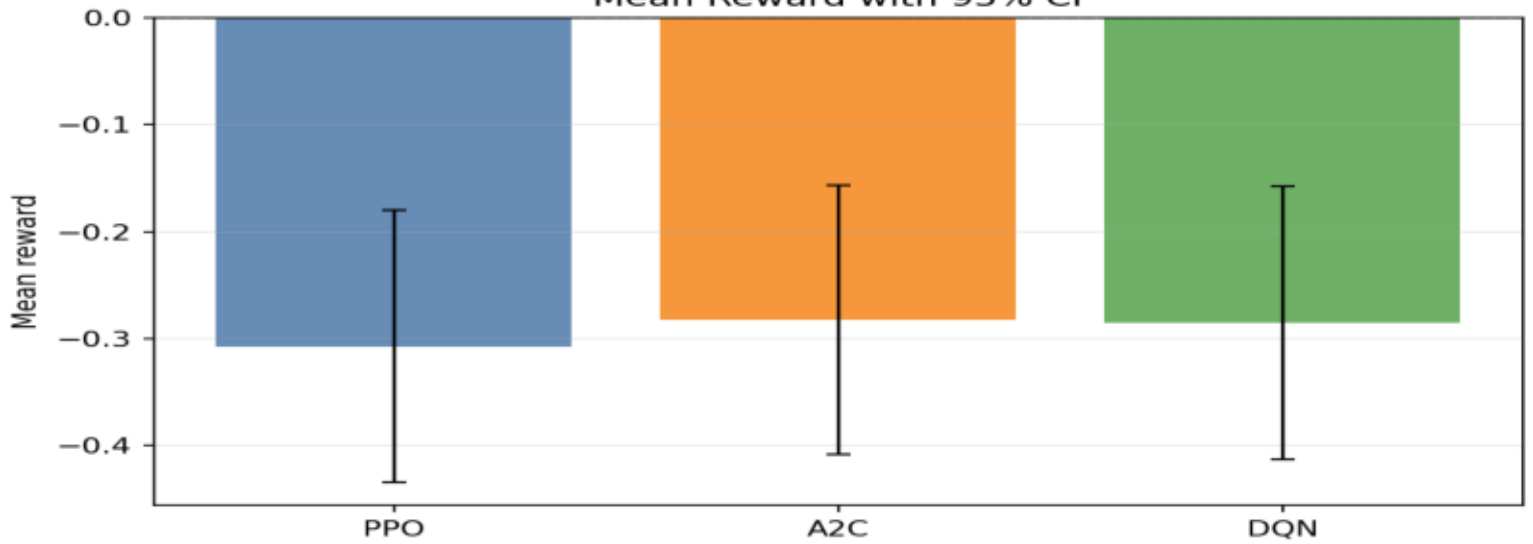
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0031	[-0.0522, +0.0583]	0.9128
PPO - A2C	-0.0251	[-0.0576, +0.0074]	0.1303
PPO - DQN	-0.0220	[-0.0726, +0.0286]	0.3940

poker - Charts

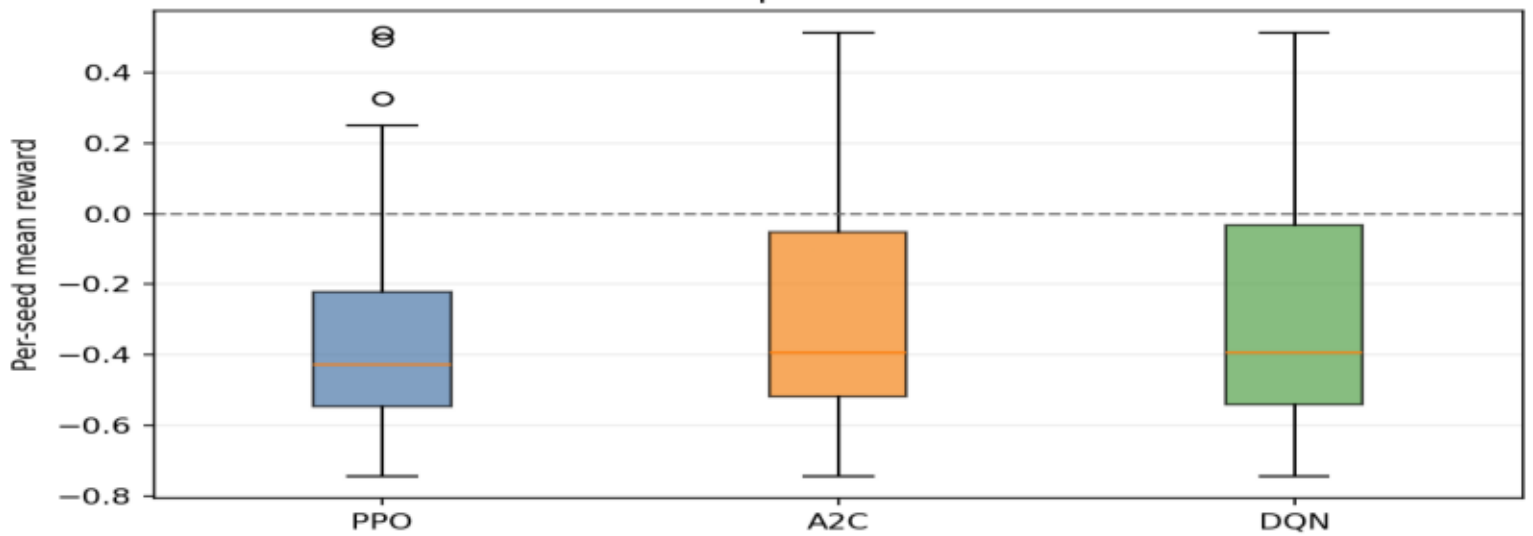
Mean Reward with 95% CI

Mean Reward with 95% CI



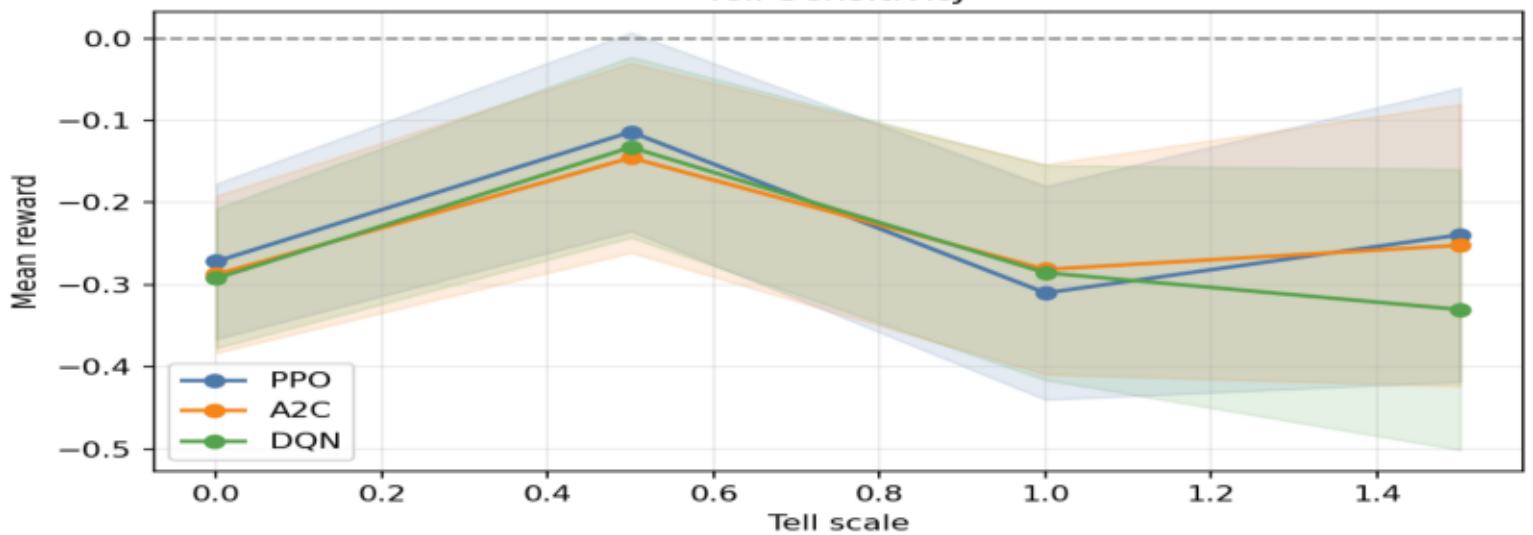
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: principal_agent
difficulty=0.62 tell_weight=0.33 profile_weight=0.34

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.2506	0.3419	0.0624	[-0.3730, -0.1283]	5.94e-05***	30
DQN	-0.2486	0.3132	0.0572	[-0.3607, -0.1365]	1.38e-05***	30
PPO	-0.2489	0.3045	0.0556	[-0.3579, -0.1399]	7.58e-06***	30

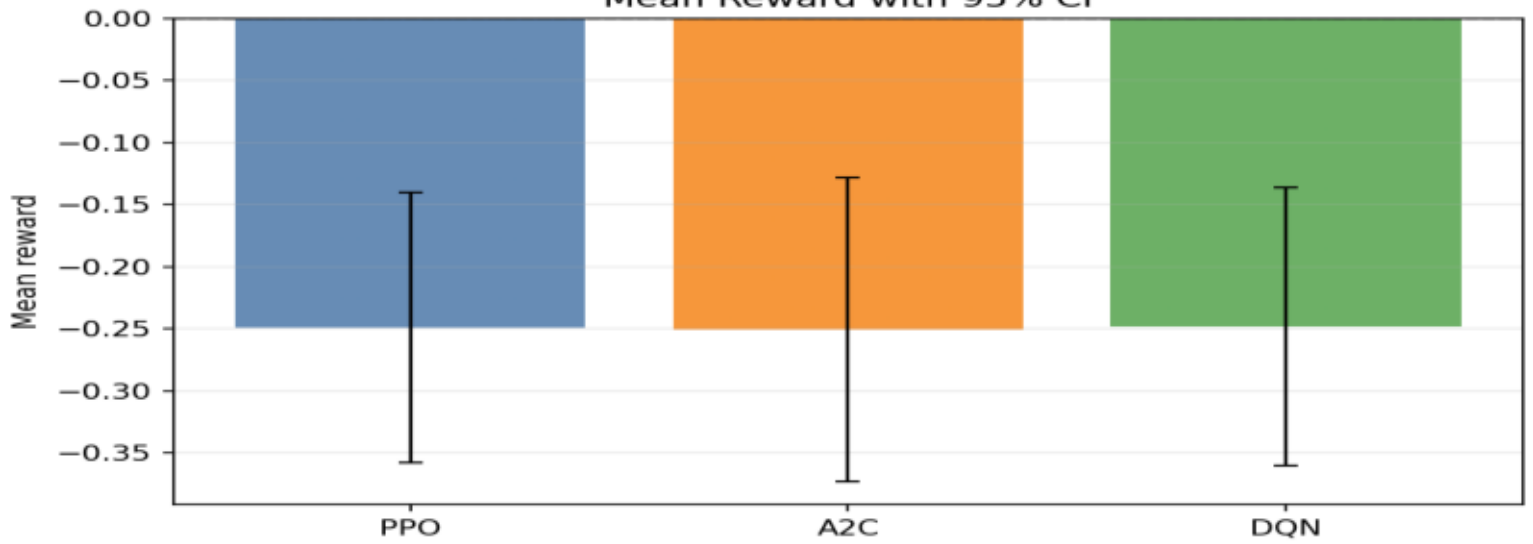
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0020	[-0.0438, +0.0398]	0.9257
PPO - A2C	+0.0017	[-0.0426, +0.0461]	0.9391
PPO - DQN	-0.0003	[-0.0463, +0.0457]	0.9910

principal_agent - Charts

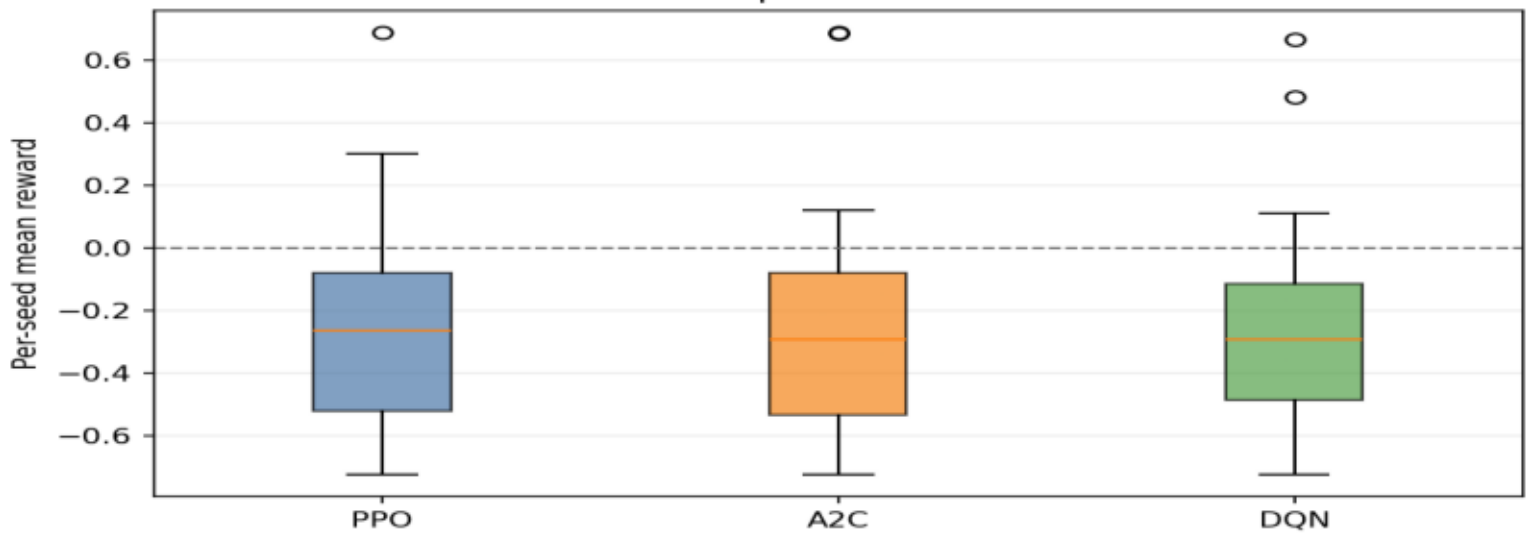
Mean Reward with 95% CI

Mean Reward with 95% CI



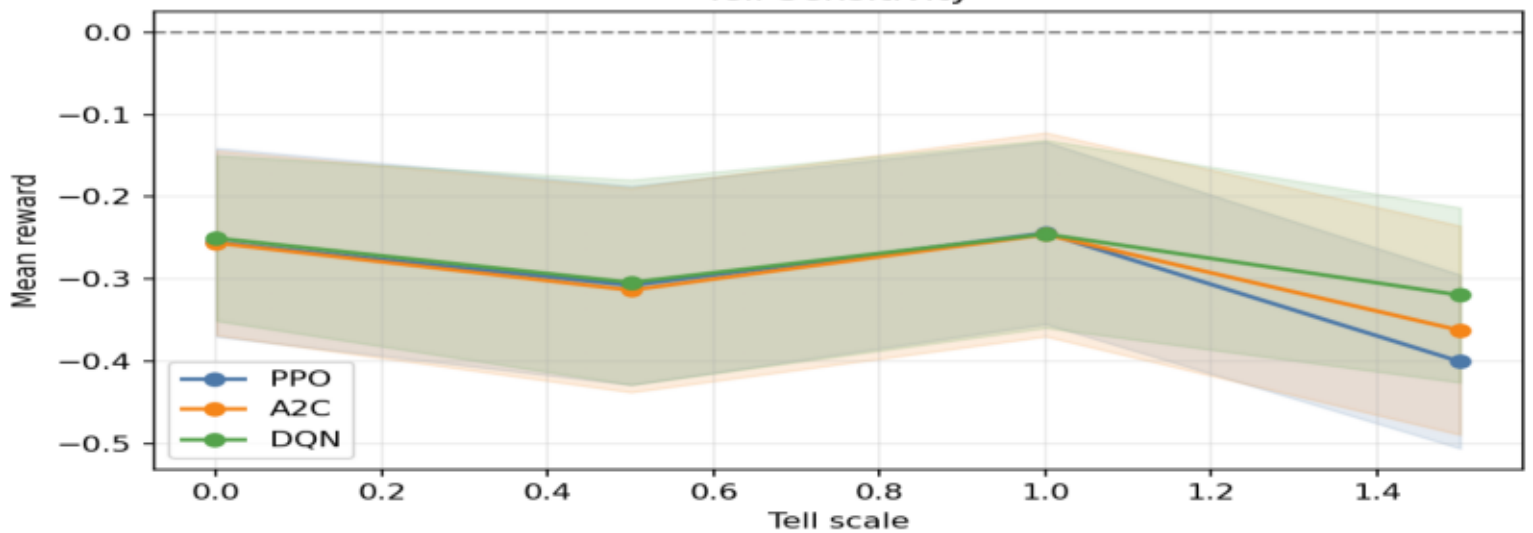
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: public_goods
difficulty=0.6 tell_weight=0.36 profile_weight=0.33

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.4075	0.3346	0.0611	[-0.5272, -0.2877]	2.56e-11***	30
DQN	-0.4007	0.2485	0.0454	[-0.4896, -0.3118]	< 1e-16***	30
PPO	-0.3819	0.3500	0.0639	[-0.5071, -0.2566]	2.29e-09***	30

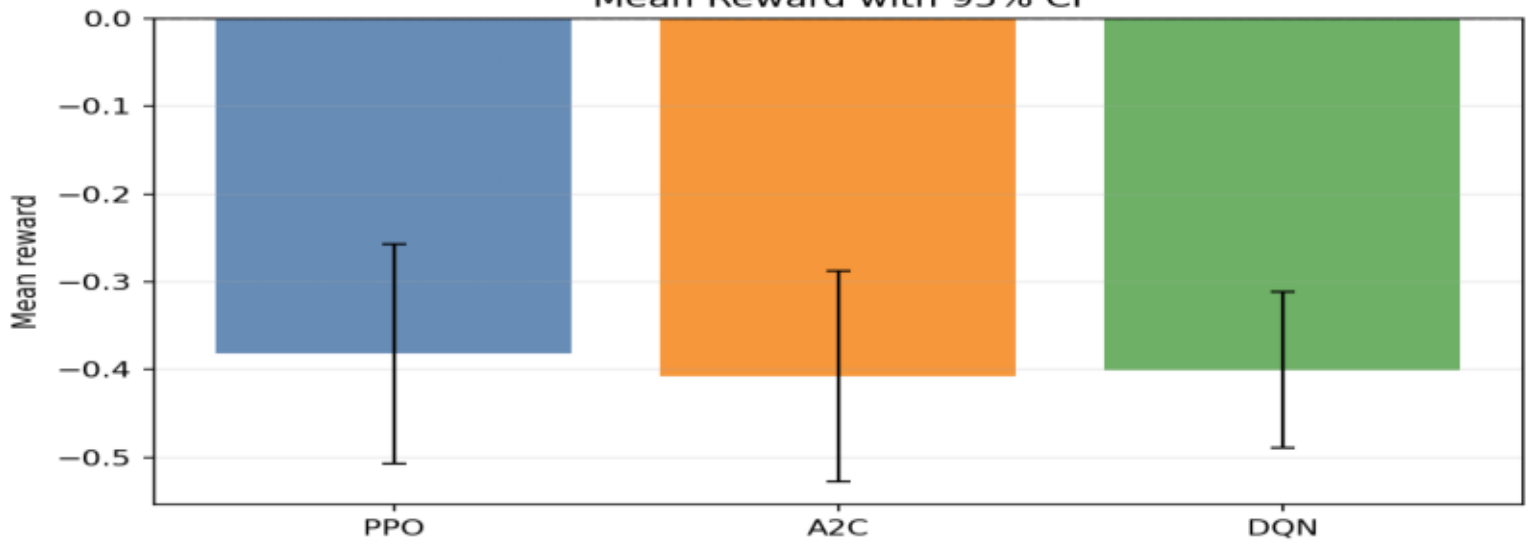
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0068	[-0.0812, +0.0677]	0.8590
PPO - A2C	+0.0256	[-0.0236, +0.0748]	0.3075
PPO - DQN	+0.0188	[-0.0617, +0.0994]	0.6465

public_goods - Charts

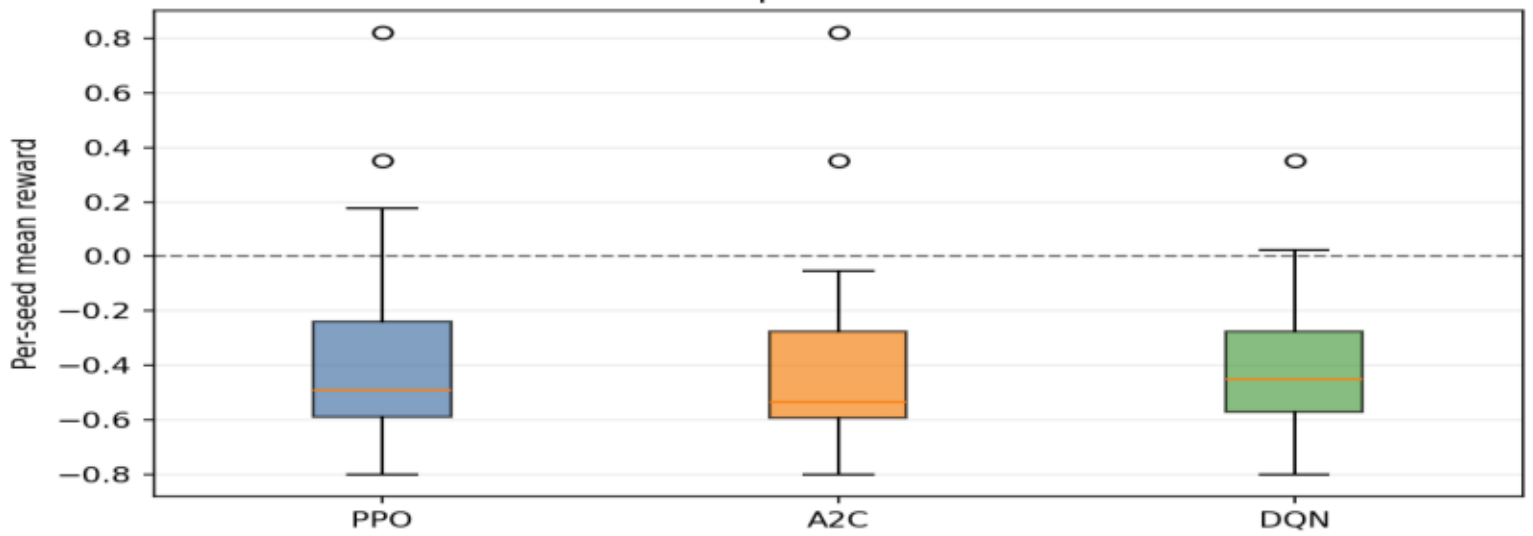
Mean Reward with 95% CI

Mean Reward with 95% CI



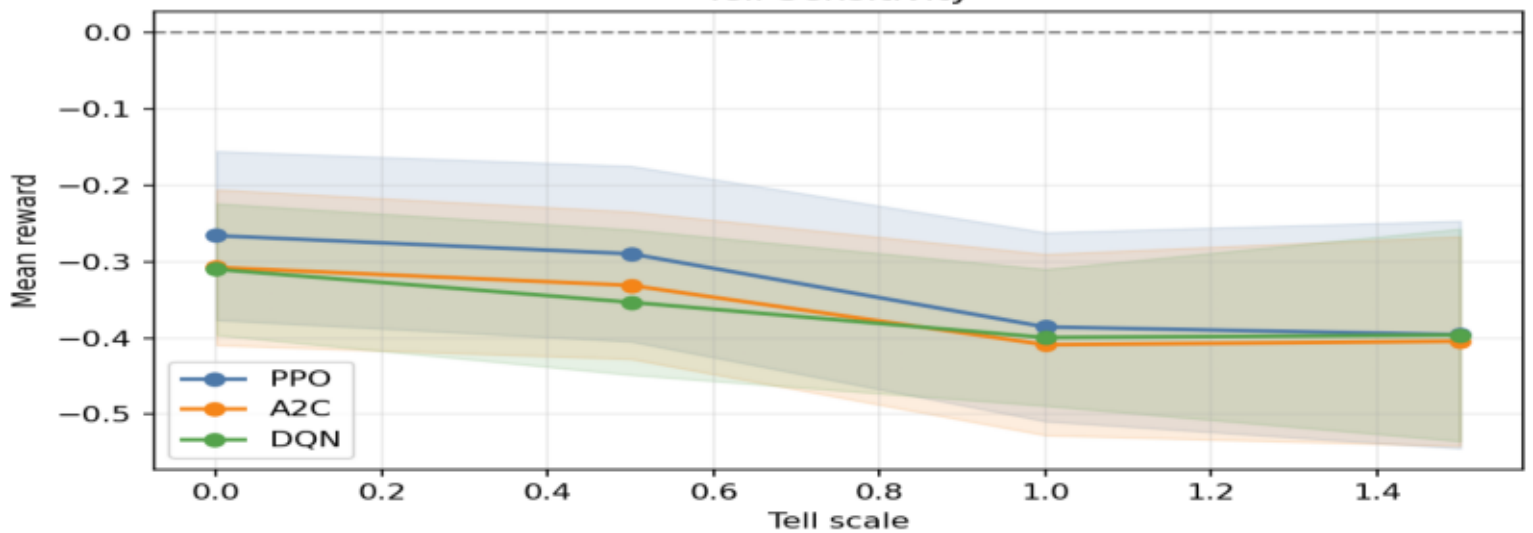
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: signaling
difficulty=0.61 tell_weight=0.39 profile_weight=0.31

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.1787	0.3212	0.0586	[-0.2937, -0.0638]	0.0023**	30
DQN	-0.1508	0.3444	0.0629	[-0.2740, -0.0275]	0.0165*	30
PPO	-0.1862	0.3781	0.0690	[-0.3216, -0.0509]	0.0070**	30

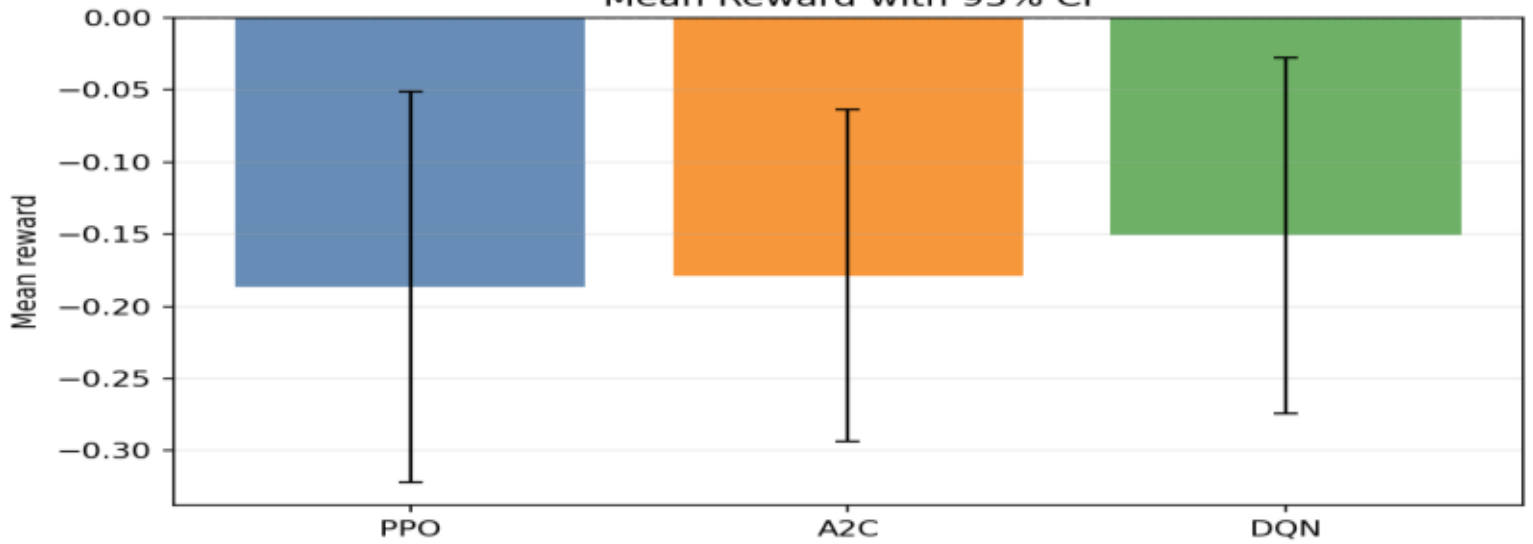
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0280	[-0.0950, +0.0391]	0.4136
PPO - A2C	-0.0075	[-0.0578, +0.0427]	0.7693
PPO - DQN	-0.0355	[-0.1111, +0.0401]	0.3576

signaling - Charts

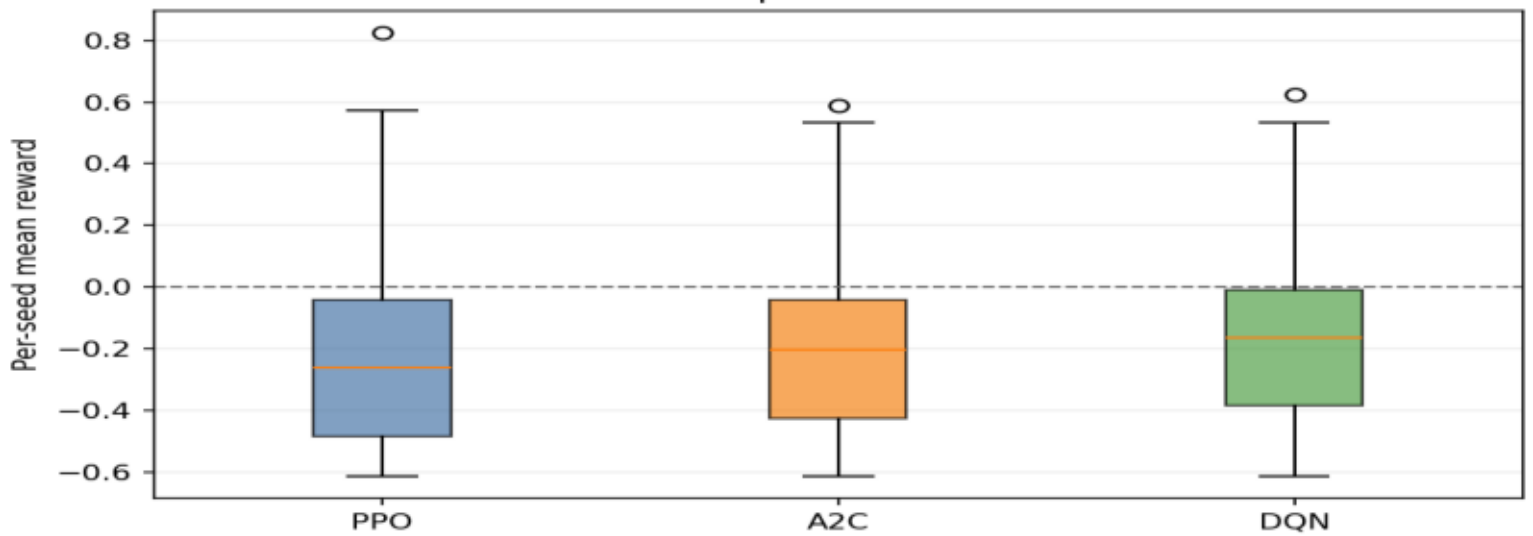
Mean Reward with 95% CI

Mean Reward with 95% CI



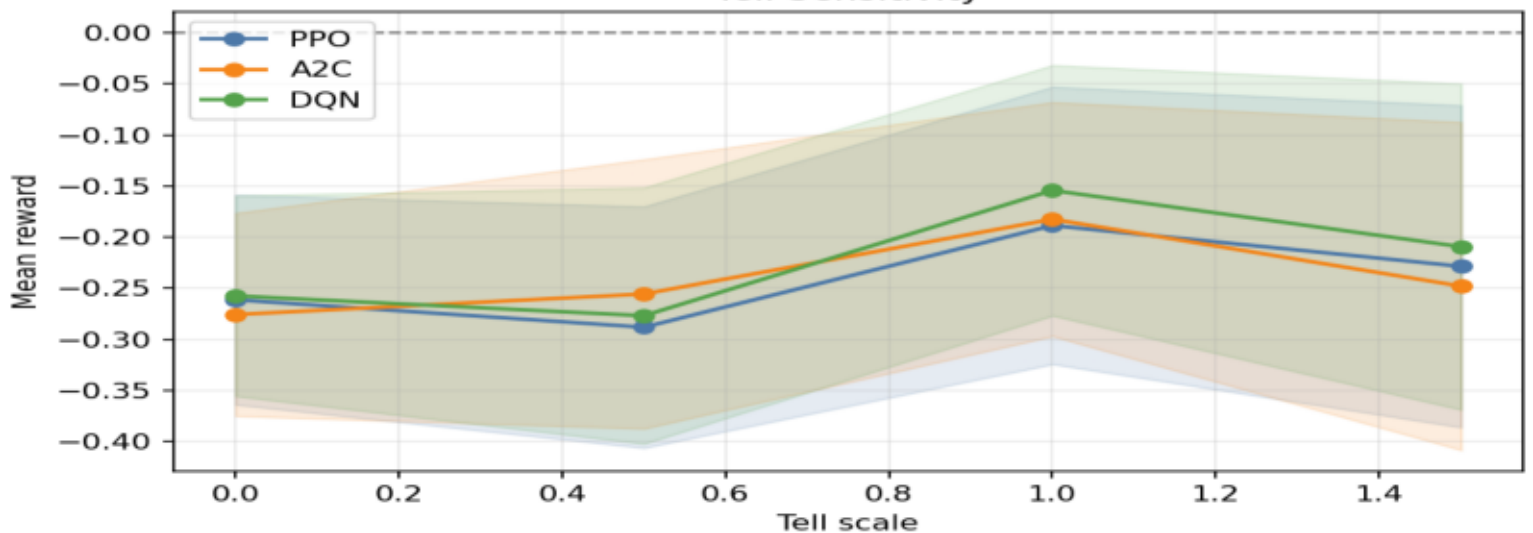
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: stag_hunt
difficulty=0.58 tell_weight=0.36 profile_weight=0.31

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	+0.2790	0.5046	0.0921	[+0.0984, +0.4595]	0.0025**	30
DQN	+0.2927	0.3900	0.0712	[+0.1532, +0.4323]	3.94e-05***	30
PPO	+0.2959	0.5049	0.0922	[+0.1152, +0.4765]	0.0013**	30

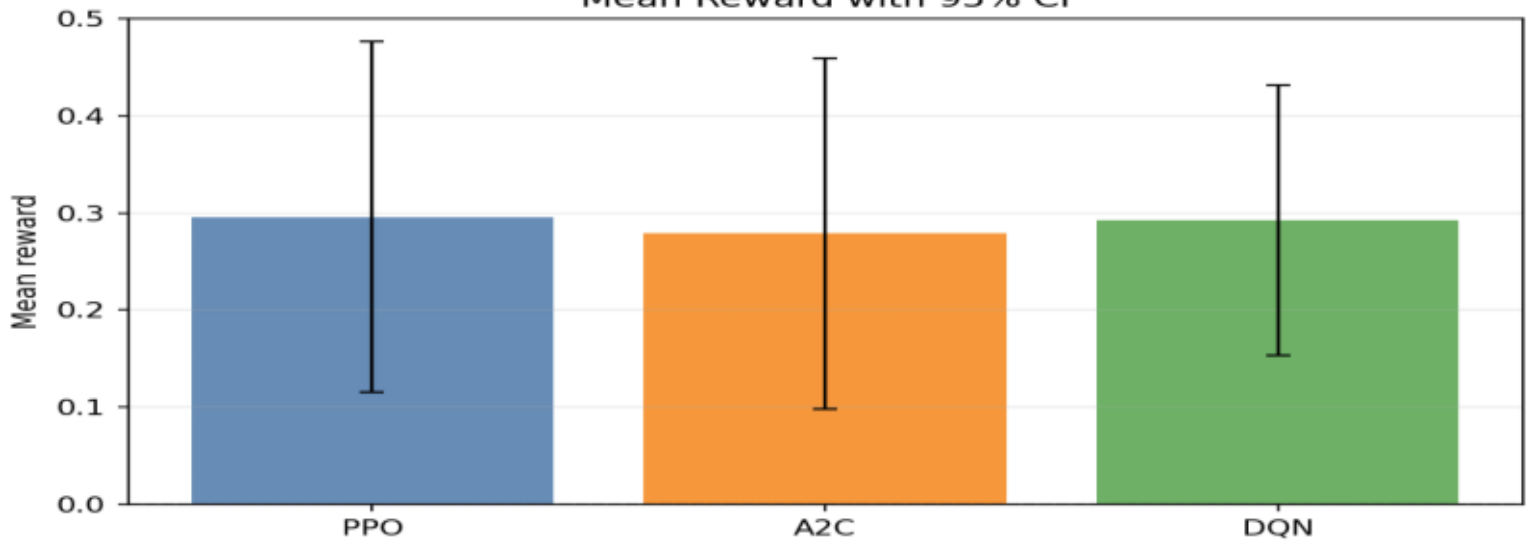
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0138	[-0.1065, +0.0790]	0.7712
PPO - A2C	+0.0169	[-0.0507, +0.0845]	0.6242
PPO - DQN	+0.0031	[-0.1123, +0.1186]	0.9574

stag_hunt - Charts

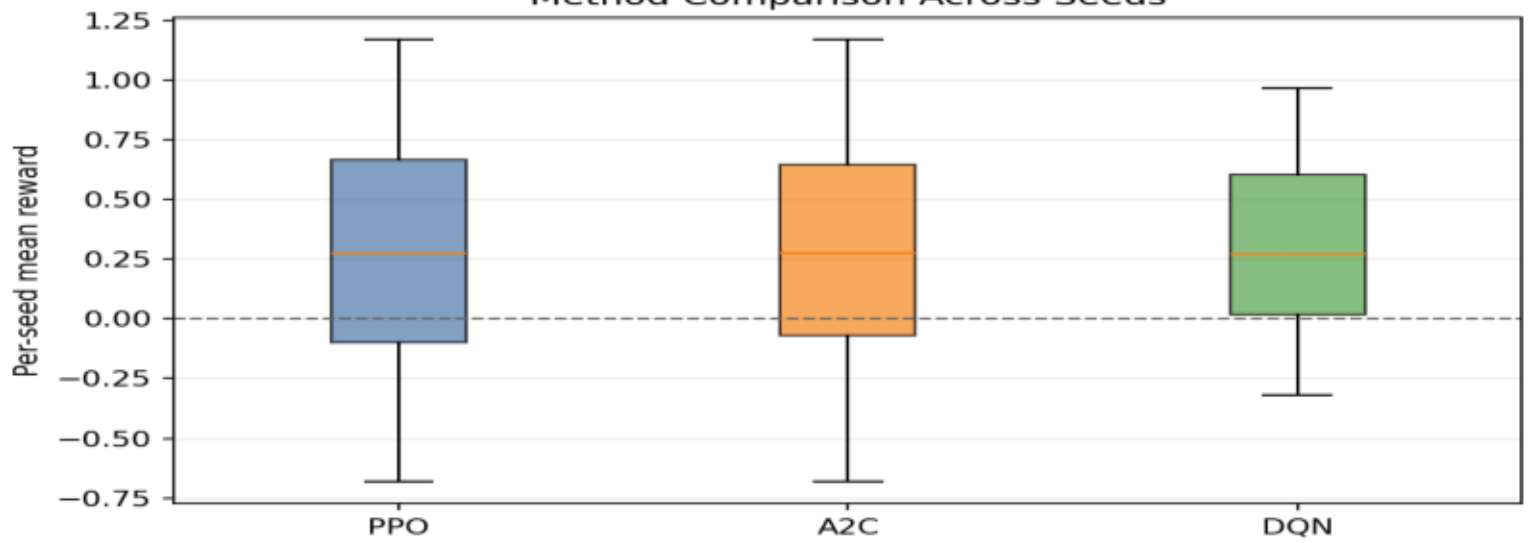
Mean Reward with 95% CI

Mean Reward with 95% CI



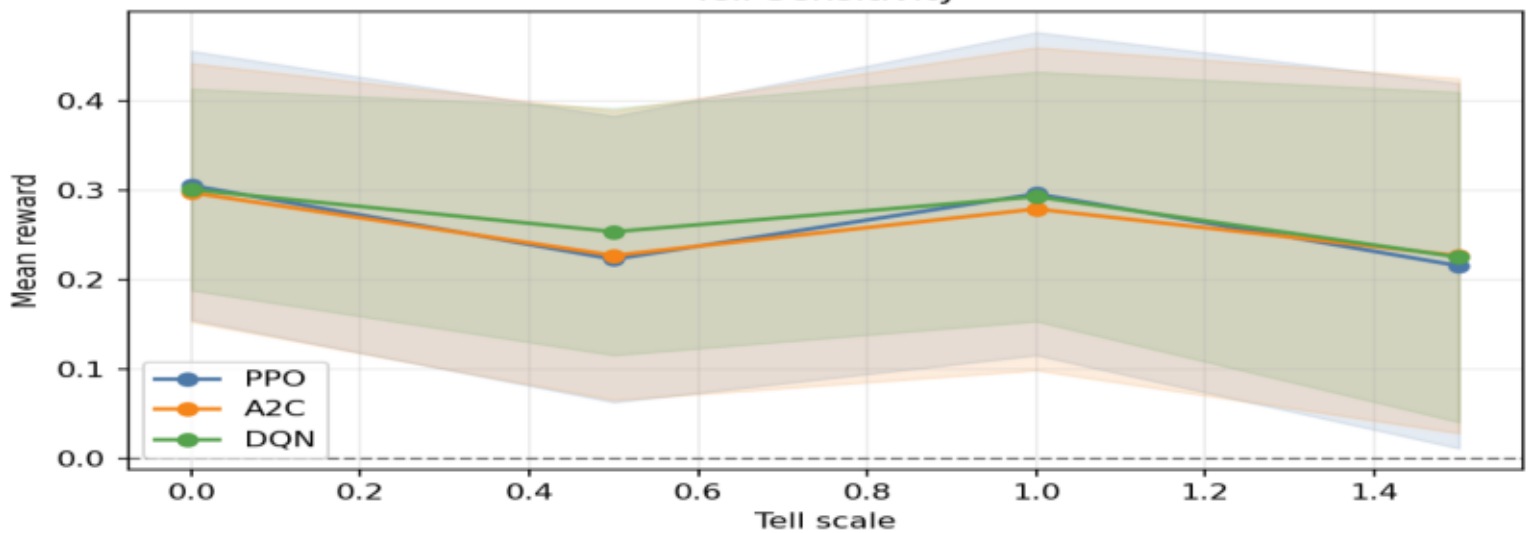
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



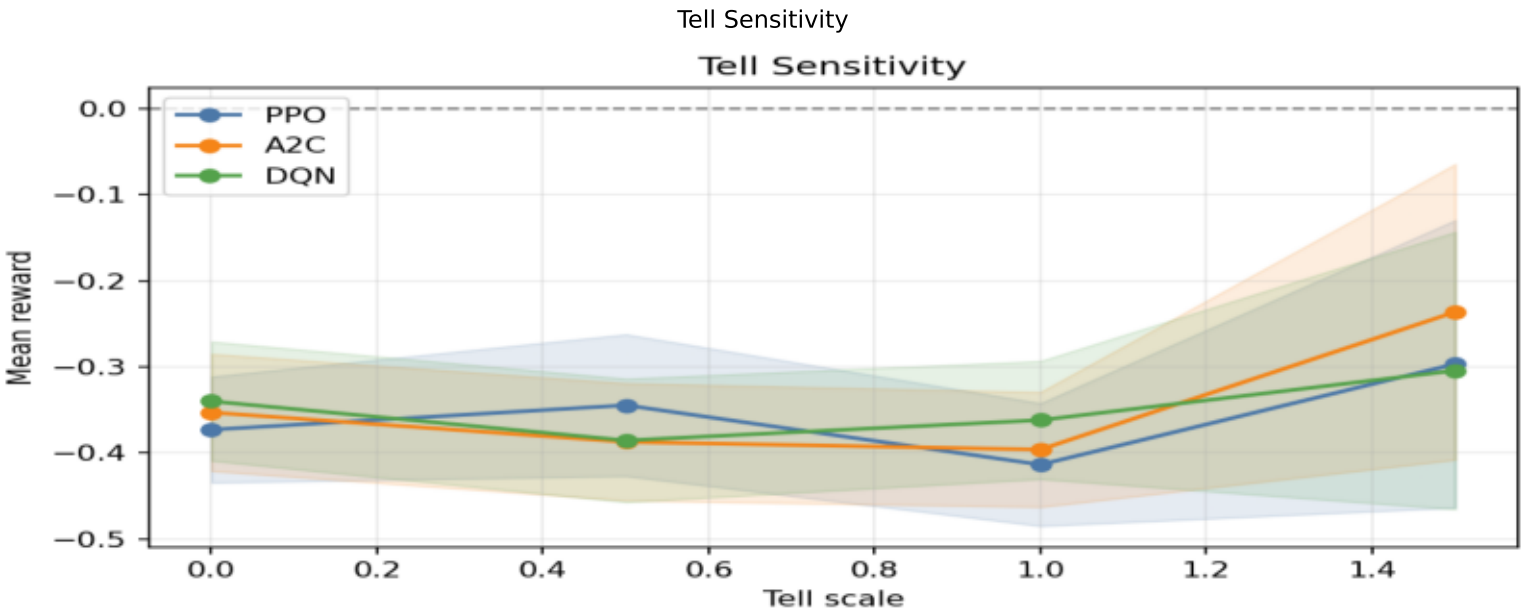
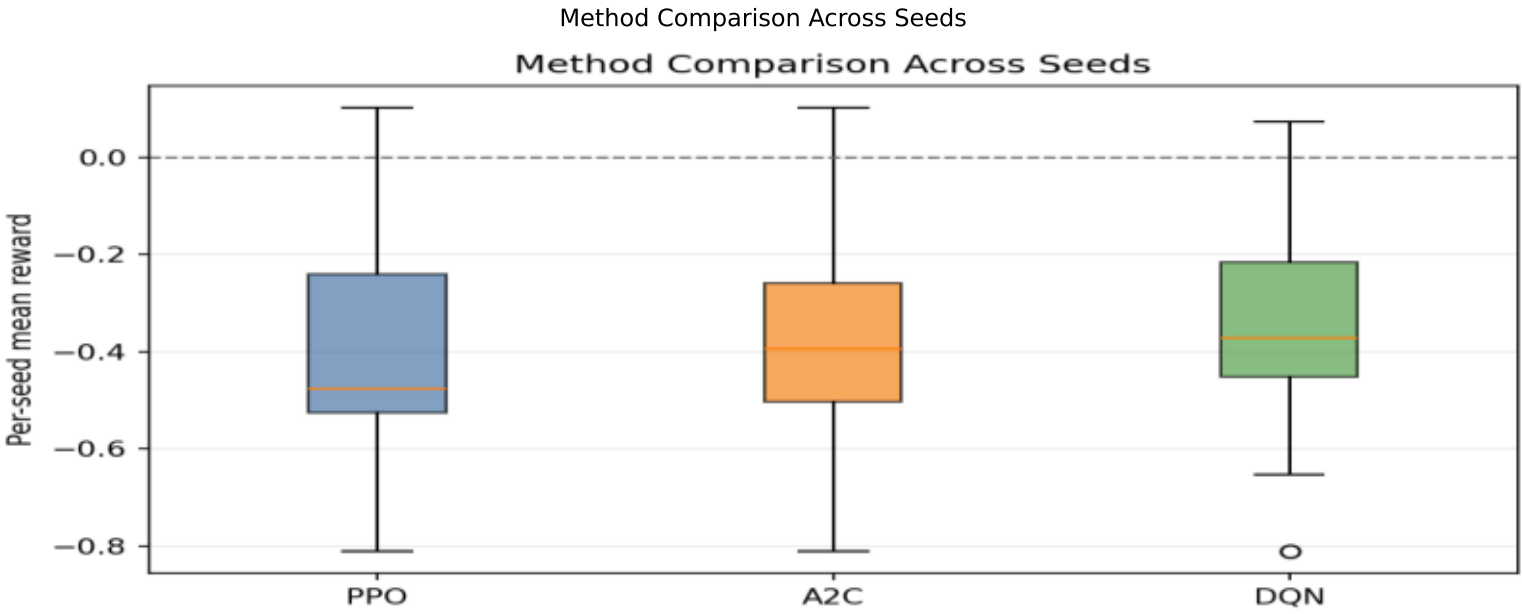
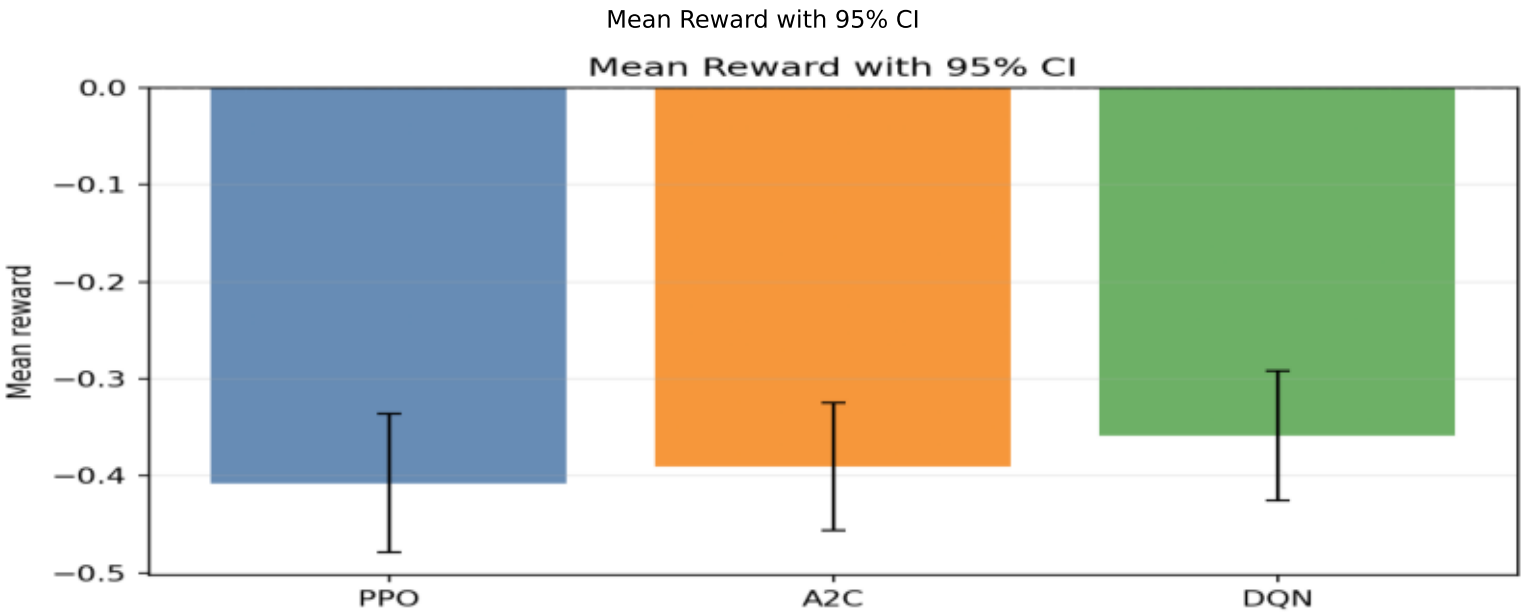
Game: tenders
difficulty=0.7 tell_weight=0.44 profile_weight=0.35

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3904	0.1837	0.0335	[-0.4562, -0.3247]	< 1e-16***	30
DQN	-0.3586	0.1866	0.0341	[-0.4254, -0.2919]	< 1e-16***	30
PPO	-0.4075	0.1984	0.0362	[-0.4785, -0.3365]	< 1e-16***	30

Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	-0.0318	[-0.0902, +0.0266]	0.2858
PPO - A2C	-0.0170	[-0.0714, +0.0373]	0.5386
PPO - DQN	-0.0489	[-0.0963, -0.0014]	0.0435*

tenders - Charts



Game: trust_game
difficulty=0.57 tell_weight=0.41 profile_weight=0.32

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3141	0.3327	0.0607	[-0.4332, -0.1951]	2.33e-07***	30
DQN	-0.3291	0.3396	0.0620	[-0.4506, -0.2076]	1.11e-07***	30
PPO	-0.3607	0.3235	0.0591	[-0.4765, -0.2450]	1.01e-09***	30

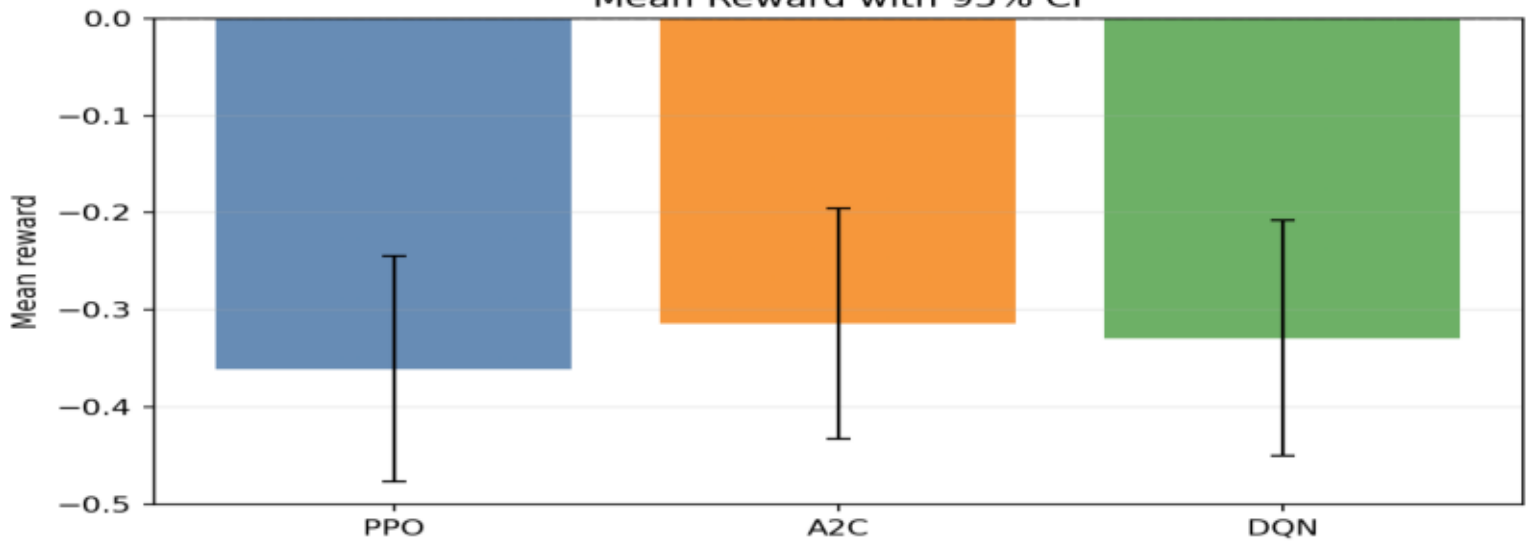
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0150	[-0.0335, +0.0635]	0.5449
PPO - A2C	-0.0466	[-0.0877, -0.0055]	0.0262*
PPO - DQN	-0.0316	[-0.0812, +0.0179]	0.2107

trust_game - Charts

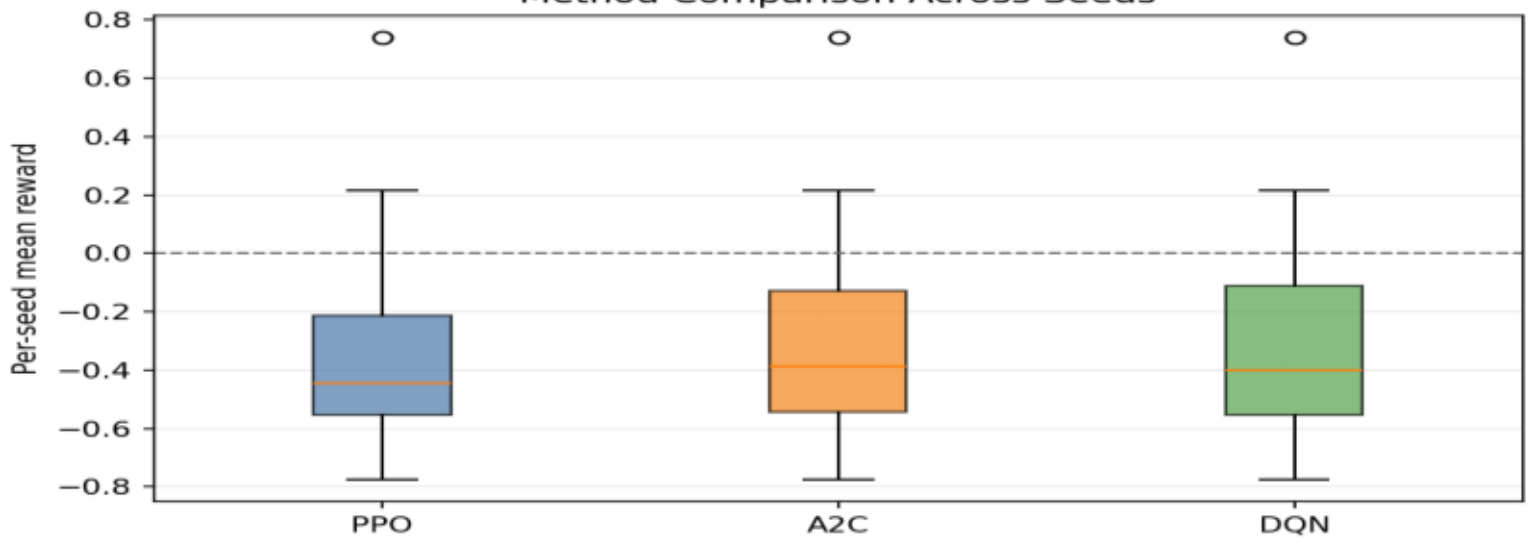
Mean Reward with 95% CI

Mean Reward with 95% CI



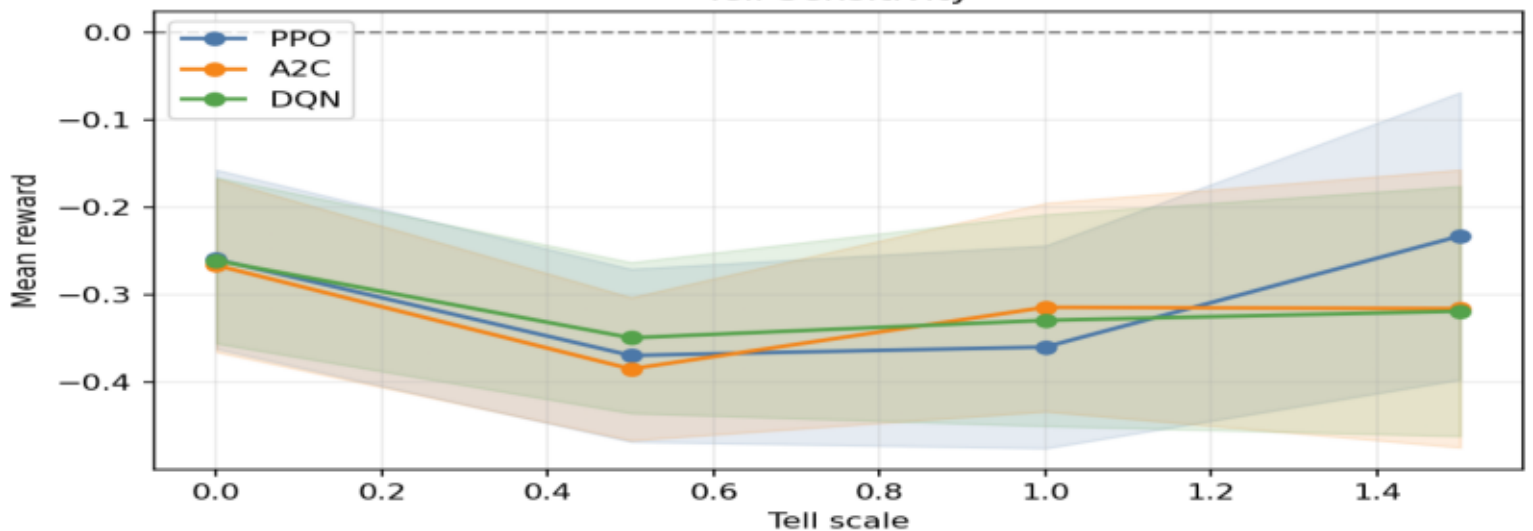
Method Comparison Across Seeds

Method Comparison Across Seeds



Tell Sensitivity

Tell Sensitivity



Game: ultimatum
difficulty=0.52 tell_weight=0.42 profile_weight=0.3

Algorithm	Mean	SD	SE	95% CI	p(vs 0)	n
A2C	-0.3515	0.3334	0.0609	[-0.4708, -0.2321]	7.79e-09***	30
DQN	-0.3642	0.3075	0.0561	[-0.4742, -0.2541]	8.78e-11***	30
PPO	-0.3881	0.3017	0.0551	[-0.4961, -0.2801]	1.85e-12***	30

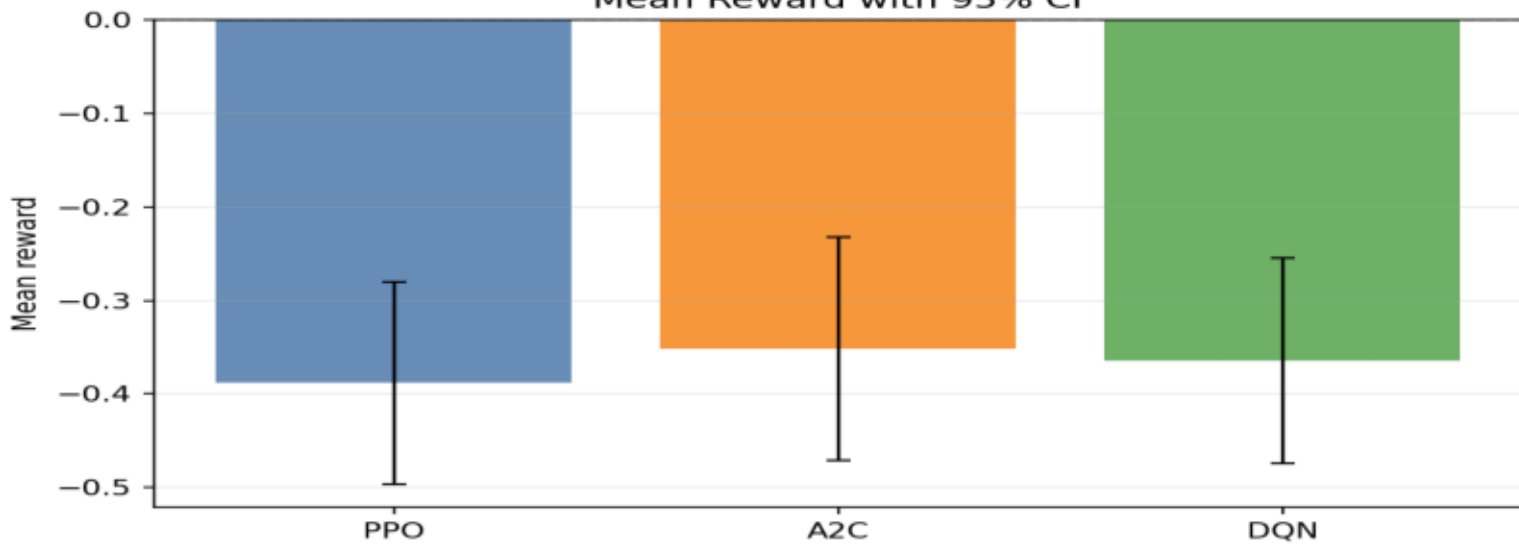
Pairwise Comparisons

Comparison	Mean Diff	95% CI	p(vs 0)
A2C - DQN	+0.0127	[-0.0370, +0.0624]	0.6162
PPO - A2C	-0.0367	[-0.1035, +0.0302]	0.2822
PPO - DQN	-0.0239	[-0.0526, +0.0048]	0.1020

ultimatum - Charts

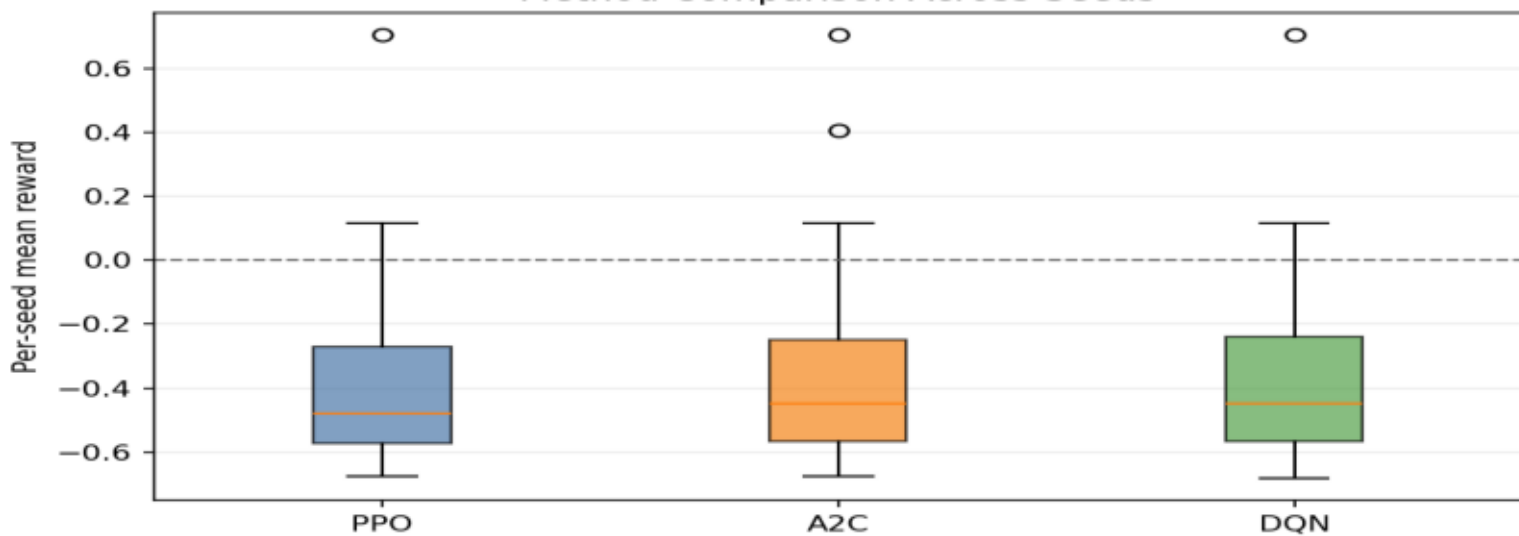
Mean Reward with 95% CI

Mean Reward with 95% CI



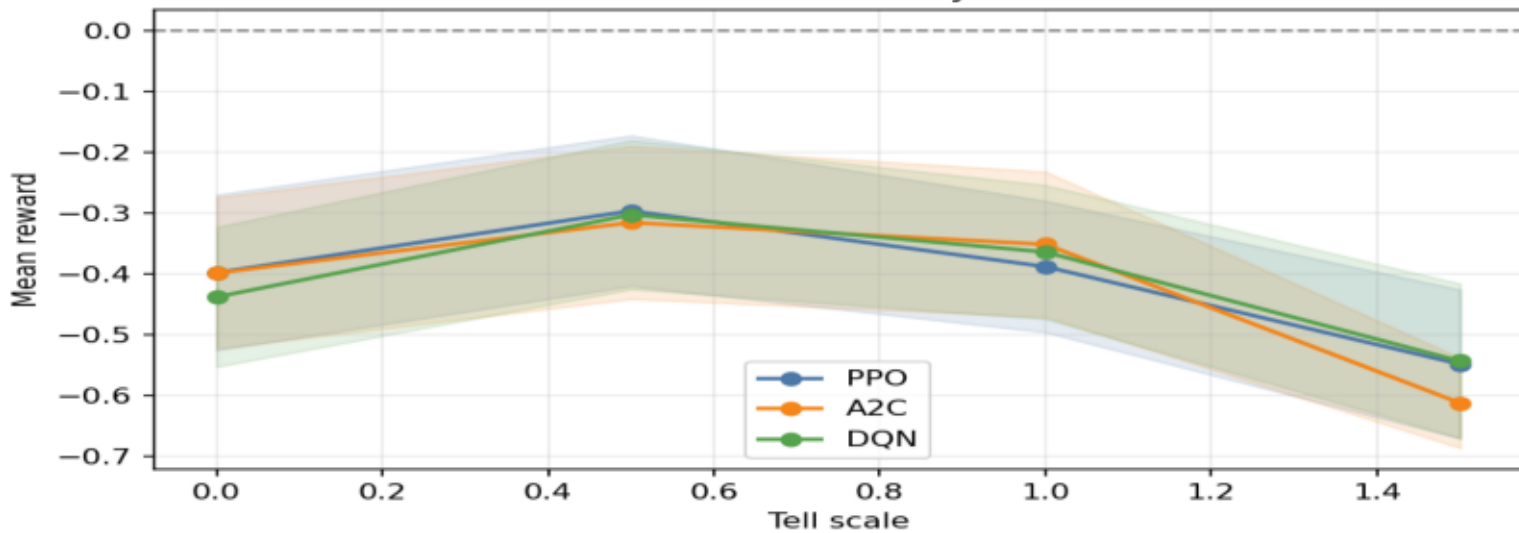
Method Comparison Across Seeds

Method Comparison Across Seeds



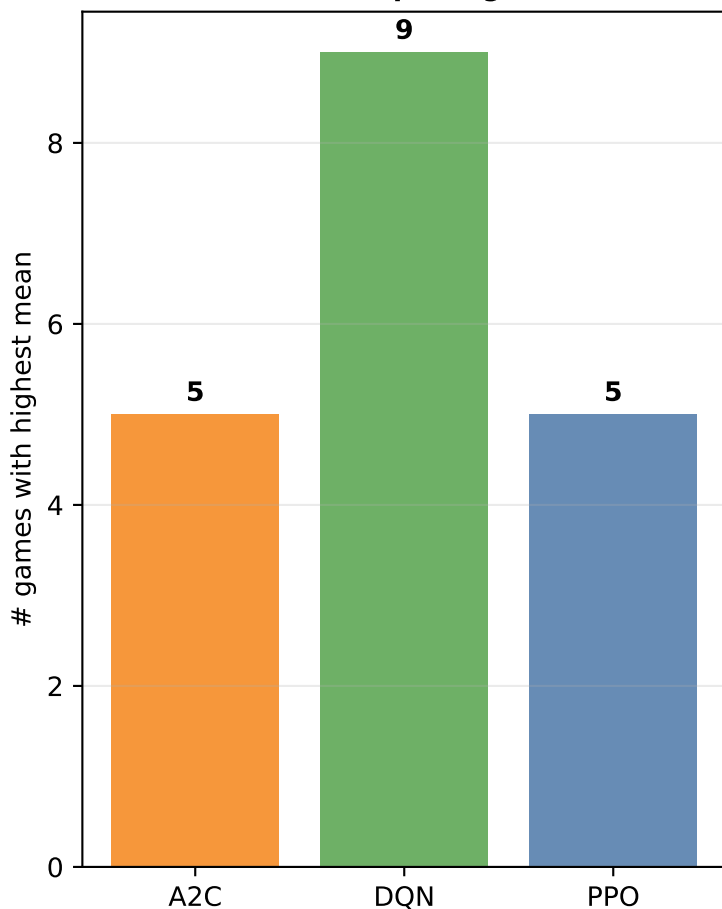
Tell Sensitivity

Tell Sensitivity

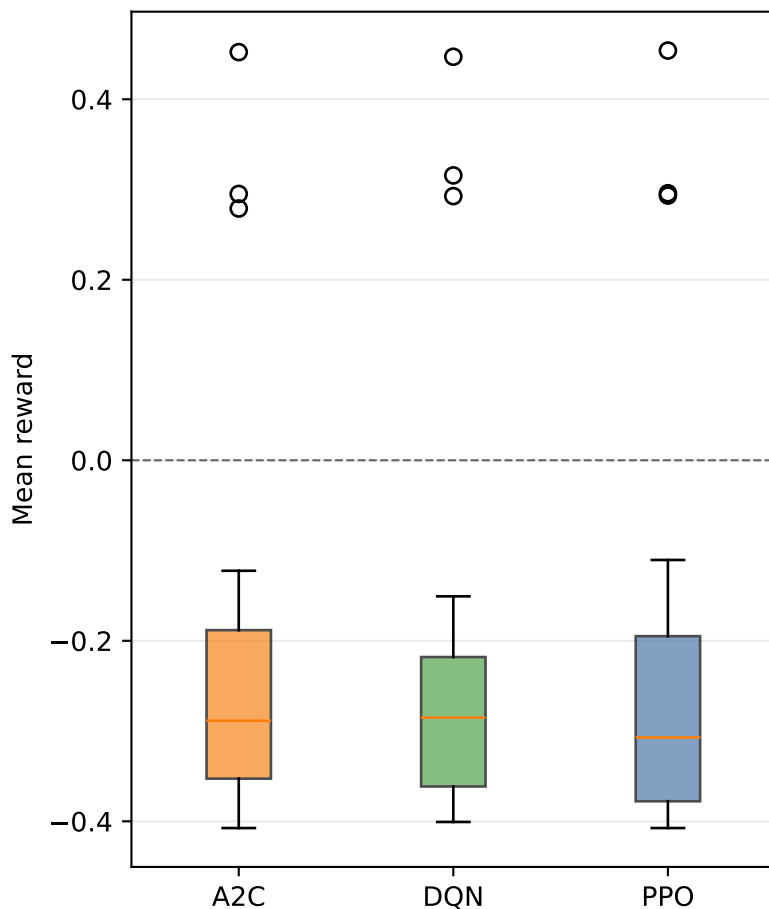


Cross-Game Comparison

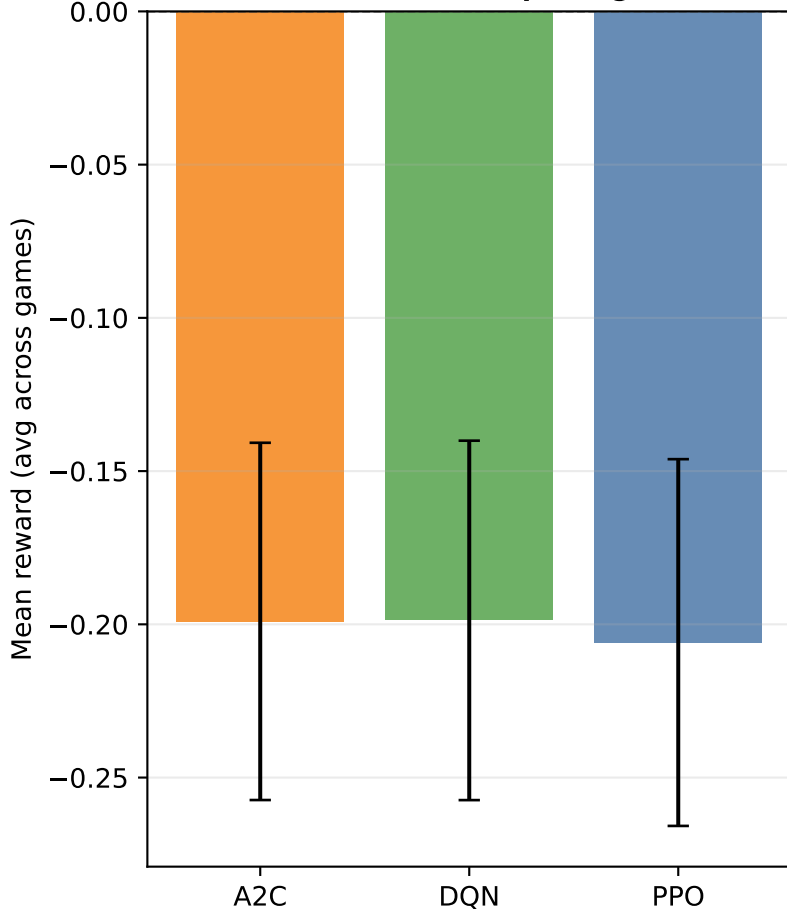
Games Won per Algorithm



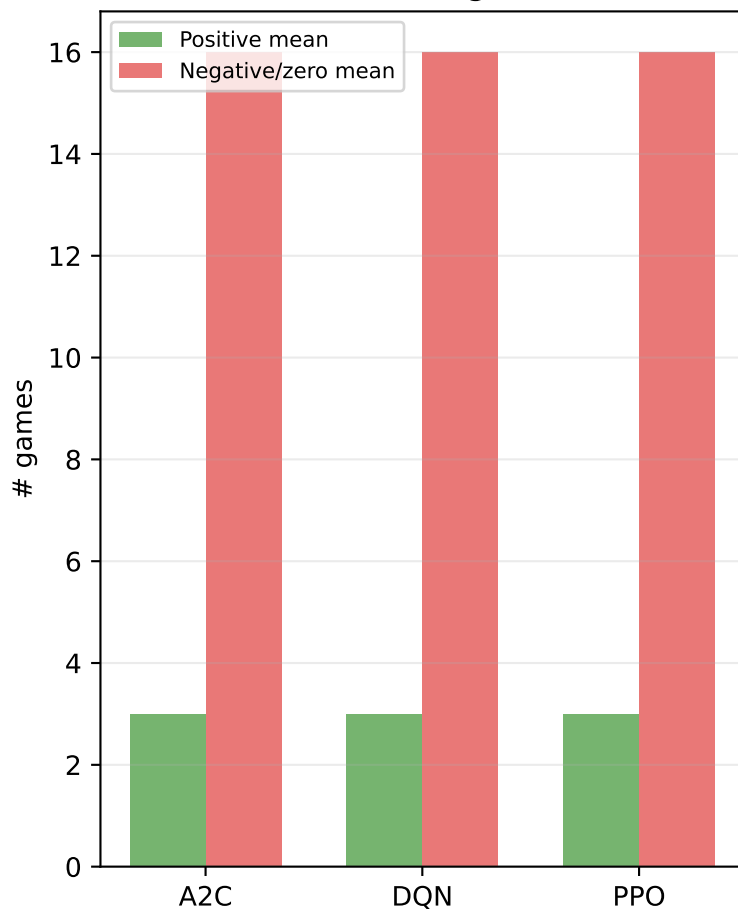
Mean Reward Distribution Across Games



Grand Mean Reward per Algorithm



Games with Positive vs Negative Mean Reward



Gap Analysis

Gap Analysis: Multi-Method Game Strategy Simulation

Date: 2026-02-18

Scope: 18 games, 3 algorithms (PPO, A2C, DQN), 30 seeds, 280 train steps, 140 eval steps

1. Performance Summary

The results are poor. Only **3 out of 18 games** produce positive mean reward. The remaining 15 games yield significantly negative rewards, confirmed at high statistical confidence.

Game	Best Algo	Mean Reward	p-value	Verdict
iterated_prisoners_dilemma	PPO	**+0.4540**	1.32e-06	Positive
hawk_dove	DQN	**+0.3156**	3.51e-05	Positive
stag_hunt	PPO	**+0.2959**	0.00133	Positive
signaling	DQN	-0.1508	0.0165	Negative
crisis_negotiations	A2C	-0.1977	0.00537	Negative
beer_distribution	DQN	-0.2242	0.000758	Negative
principal_agent	DQN	-0.2486	1.38e-05	Negative
market_for_lemons	DQN	-0.2581	3.13e-05	Negative
poker	A2C	-0.2820	1.13e-05	Negative
chess	DQN	-0.3123	2.28e-13	Negative
trust_game	A2C	-0.3141	2.33e-07	Negative
auctions	PPO	-0.3370	5.70e-10	Negative
cournot	A2C	-0.3401	8.30e-11	Negative
interrogation	DQN	-0.3478	8.08e-11	Negative
ultimatum	A2C	-0.3515	7.79e-09	Negative
tenders	DQN	-0.3586	0.00e+00	Negative
bertrand	DQN	-0.3741	9.06e-13	Negative
public_goods	PPO	-0.3819	2.29e-09	Negative

Algorithm win counts: PPO=4, DQN=9, A2C=5. DQN wins the most games, but this is misleading -- it simply loses slightly less badly in most cases.

Honest bottom line: The system does not work for the majority of games. Positive results on 3 games are real but likely explained by those games being trivially simple (see Section 6). For 15/18 games the agent performs worse than doing nothing.

2. Method Limitations

All three "algorithms" are severely stripped down relative to their real counterparts.

2.1 Linear Function Approximation Only

Every learner uses a single linear layer ($W @ state + b$) with no hidden layers, no activation functions, and no neural network of any kind:

- **DQN** (`methods.py:69-70`): `q_w` is `(n_actions, state_dim)`, `q_b` is `(n_actions,)`.

Q-values are $q_w @ \text{state} + q_b$.

- **A2C** (`methods.py:131-134`): Actor is $\text{actor}_w @ \text{state} + \text{actor}_b$; critic is $\text{value}_w @ \text{state} + \text{value}_b$. Both linear.
- **PP0** (`methods.py:178`): Inherits from A2C; same linear architecture. The clip mechanism modulates the gradient scale but still operates on a linear policy.

A linear function approximator cannot learn nonlinear decision boundaries. For any game where the optimal action depends on interactions between state features (which is most games), the learner is fundamentally incapable of representing the correct policy. This is the single biggest limitation.

2.2 DQN Missing Core Components

The DQN implementation (`methods.py:59-121`) is missing two components that made the original DQN work:

- **No experience replay buffer.** The agent learns from each transition once, immediately, and discards it. This means highly correlated sequential samples, which destabilizes learning.
- **No target network.** The same weights are used for both Q-value estimation and target computation. The target shifts with every update, creating a moving-target problem.

Without these, this is not really DQN -- it is online linear Q-learning with epsilon-greedy exploration.

2.3 PPO Clip Is Approximate

PPO (`methods.py:186-224`) computes a ratio $\text{new_prob} / \text{old_prob}$ and clips it, but the underlying policy update is still a scaled linear gradient step. Real PPO runs multiple epochs of minibatch updates on a collected trajectory; this implementation does a single online update per step. The clip mechanism here constrains the gradient magnitude but does not provide the trust-region-like guarantees of actual PPO.

2.4 A2C Missing Entropy Bonus

The A2C implementation (`methods.py:123-175`) has no entropy regularization term. Entropy bonuses are critical for preventing premature convergence to deterministic policies. Without it, A2C quickly collapses to greedy behavior before exploring the action space sufficiently.

2.5 Training Budget Is Extremely Short

All games train for exactly **280 steps**. For context:

- The original DQN paper trained for 50 million frames.
- Standard RL benchmarks use thousands to millions of environment steps.
- 280 steps with a linear model means the weight matrix receives 280 gradient updates total. For a game like tenders with 12 actions and 26-dimensional state (13 base + 8 tell + 5 profile), that is 280 updates to a (12, 26) weight matrix -- roughly 0.9 updates per parameter on average.

This is not enough training for any method to learn meaningful behavior on complex tasks.

3. Simulation Limitations

3.1 Reward Function Is Classification-Based, Not Game-Theoretic

The environment (``eval_harness.py:189-193``) computes reward as:

```
```python
classification_reward = 1.0 if action == optimal_action else (-0.35 - 0.15 * margin)
signal_reward = 0.25 * expected[action]
reward = classification_reward + signal_reward + noise[action]
```
```

This is dense oracle-based supervision: the agent gets +1 for matching the oracle's preferred action and a penalty proportional to the regret margin otherwise. This is closer to supervised classification than reinforcement learning. Real game-theoretic payoffs (e.g., Nash equilibrium rewards, actual prisoner's dilemma payoff matrices) are not used.

The "optimal action" is defined by ``np.argmax(expected_rewards)``, where ``expected_rewards`` is itself a synthetic function of the state, action weights, tell features, and profile. The agent is learning to imitate a randomly-seeded linear oracle, not to play a real game.

3.2 Environments Are Synthetic and Parametric

The ``LightweightGameSimulator`` (``eval_harness.py:91-205``) generates states and rewards entirely from ``game_specs.py`` parameters. Key observations:

- **Base states** are RNG-generated with sinusoidal/cosinusoidal modulation (``eval_harness.py:128-139``). They do not represent real game states (board positions, card distributions, auction bids, etc.).
- **Tell features** are synthetic from ``MockVideoFeed`` (``mock_video_feed.py:40-115``), producing 8-dimensional vectors (gaze_aversion, blink_rate, jaw_tension, etc.) that are deterministic functions of profile traits and step index. These are not actual behavioral signals.
- **The action-reward mapping** uses a random weight matrix (``eval_harness.py:113-114``) scaled by difficulty, tell_weight, and profile_weight. The "game" is: match the argmax of a noisy linear function. Every game uses the same structure; they differ only in dimensionality, noise level, and scaling parameters.

This means the 18 "games" are 18 parameterizations of the same synthetic environment, not 18 different game-theoretic settings.

3.3 No Opponent Modeling or Multi-Agent Interaction

All evaluations are single-agent against a static environment. There is:

- No opponent that adapts, learns, or responds to the agent's strategy.
- No self-play or population-based training.
- No multi-agent dynamics whatsoever.

This is especially problematic for games where opponent interaction is definitional (prisoner's dilemma, hawk-dove, stag hunt, negotiation games, auctions). The agent is not "playing" these games in any meaningful sense.

3.4 Opponent Profiles Are Fixed and Deterministic

The `choose_profile` function (`opponent_intel.py:59-63`) deterministically selects from 5 fixed profiles (balanced, aggressive, tight, tricky, composed) based on a hash of game name and seed. The profile does not change during an episode. A real opponent would vary behavior dynamically.

4. Statistical Concerns

4.1 Sample Size (30 Seeds)

With 30 seeds per game per algorithm, the 95% confidence intervals are moderately wide. Representative examples from the data:

| Game | Algo | Mean | CI95 Width | SD |
|----------------------------|------|--------|------------|-------|
| iterated_prisoners_dilemma | PP0 | +0.454 | 0.368 | 0.514 |
| hawk_dove | DQN | +0.316 | 0.299 | 0.418 |
| stag_hunt | A2C | +0.279 | 0.361 | 0.507 |
| bertrand | DQN | -0.374 | 0.205 | 0.287 |
| tenders | DQN | -0.359 | 0.134 | 0.187 |

For the 3 positive games, CI widths range from 0.30 to 0.37, meaning the true mean could be anywhere from modestly positive to strongly positive. This is adequate for directional conclusions but not for precise effect-size estimation. For the negative games, the CIs are tighter (0.13-0.25) because SDs are smaller, making the "significantly negative" conclusion robust.

30 seeds is sufficient to detect the direction of effect but insufficient for fine-grained algorithm comparisons.

4.2 Tiny P-Values Do Not Mean Good Performance

Most p-values are astronomically small (e.g., $p=0$ for tenders, $p=9e-13$ for bertrand). This is a statistical trap: these p-values test "is the mean different from zero?" not "is the performance good?" The answer is yes, the performance is reliably and significantly **bad**. The small p-values simply confirm that the negative bias is real and reproducible, not random.

For the 3 positive games, the p-values ($1.3e-6$, $3.5e-5$, $1.3e-3$) confirm the positive bias is also real, but the effect sizes (+0.29 to +0.45) are modest.

4.3 No Multiple Testing Correction

18 games are tested simultaneously. Without Bonferroni or FDR correction:

- Bonferroni-adjusted alpha at 0.05: $0.05/18 = 0.00278$
- Under Bonferroni, signaling ($p=0.0165$) and crisis_negotiations ($p=0.00537$) would lose significance.
- The 3 positive games all survive Bonferroni correction.
- The strongly negative games ($p < 1e-5$) all survive any reasonable correction.

The lack of correction does not change the main conclusions but is a methodological gap.

4.4 Pairwise Algorithm Comparisons Are Uninformative

Looking at pairwise differences across the metrics data:

| Game | Comparison | Mean Diff | p-value |
|----------------------------|------------|-----------|---------|
| iterated_prisoners_dilemma | PP0 - DQN | +0.0069 | 0.691 |
| iterated_prisoners_dilemma | PP0 - A2C | +0.0017 | 0.856 |
| hawk_dove | PP0 - DQN | -0.0223 | 0.599 |
| bertrand | PP0 - DQN | -0.0079 | 0.810 |
| tenders | PP0 - DQN | -0.0489 | 0.044 |

Almost all pairwise p-values are non-significant ($p > 0.05$). The three algorithms perform nearly identically on most games. This makes sense: they are all linear models with similar capacity. There is no meaningful algorithm selection happening; the "best" algorithm per game is determined by noise, not by genuine algorithmic advantage.

5. Missing Features

5.1 No Hyperparameter Tuning

All games use identical hyperparameters hardcoded in `methods.py`:

| Parameter | DQN | A2C | PP0 |
|-------------------|-----------|-------------------------------|-------------------------------|
| Learning rate | 0.035 | 0.016 (actor), 0.024 (critic) | 0.014 (actor), 0.022 (critic) |
| Gamma | 0.96 | 0.96 | 0.96 |
| Epsilon start/end | 0.22/0.02 | -- | -- |
| Clip ratio | -- | -- | 0.20 |
| Temperature | -- | 1.0 | 1.0 |

No game-specific tuning is performed. Games with 2 actions (hawk_dove, stag_hunt, IPD) and games with 12 actions (tenders, interrogation) use identical learning rates and exploration parameters. This one-size-fits-all approach almost certainly hurts performance on games at the extremes.

5.2 No Random Baseline

There is no comparison to a uniform-random policy. A random agent choosing actions uniformly would get the classification reward (+1) with probability $1/n_{\text{actions}}$ per step. For 2-action games, random gets +1 half the time and roughly -0.35 half the time, yielding $\sim +0.325$ expected reward. This means **the positive results on 2-action games may not even beat random**. Without a random baseline, we cannot assess whether the learners are doing anything useful.

5.3 No Oracle Baseline

There is no comparison to an agent that always picks the optimal action. Since the environment provides oracle information (`info["optimal_action"]`), computing the oracle upper bound is

trivial. Without it, we cannot assess what fraction of possible performance the learners capture.

5.4 No Curriculum Learning or Staged Training

All 280 training steps use the same environment difficulty. No curriculum (easy-to-hard), no warm-start, no staged learning.

5.5 No Architecture Search

The linear architecture was chosen, not validated. No comparison to:

- 1-hidden-layer MLP
- Tabular methods (for small action spaces)
- Simple lookup tables
- Nearest-neighbor approaches

5.6 No Model Selection Framework

The "best algorithm" is selected post-hoc by comparing means. There is no cross-validation, no holdout set, no model selection criterion. The 30 evaluation seeds are used both for selection and for reporting, which inflates the best-of-3 mean through selection bias.

6. Why Only 3 Games Succeed

The 3 positive games share critical properties:

| Game | n_actions | difficulty | base_state_dim | reward_noise |
|----------------------------|-----------|------------|----------------|--------------|
| iterated_prisoners_dilemma | **2** | **0.56** | 9 | 0.14 |
| hawk_dove | **2** | **0.57** | 8 | 0.15 |
| stag_hunt | **2** | **0.58** | 8 | 0.14 |

Compare to the worst-performing games:

| Game | n_actions | difficulty | base_state_dim | reward_noise |
|---------------|-----------|------------|----------------|--------------|
| tenders | 12 | 0.70 | 13 | 0.16 |
| bertrand | 9 | 0.64 | 10 | 0.15 |
| public_goods | 11 | 0.60 | 10 | 0.15 |
| interrogation | 12 | 0.71 | 12 | 0.18 |

The pattern is clear:

1. **All 3 successes are 2-action games.** With only 2 actions, the classification reward gives +1 on the correct action and ~-0.35 on the wrong action. A linear model with 280 training steps has a reasonable chance of learning which side of a hyperplane each state falls on. For 9-12 action games, the correct action is a 1-of-N classification problem where the linear model must partition a high-dimensional space into many more regions.
2. **The 3 successes have the lowest difficulty scores** (0.56-0.58). The difficulty parameter

directly scales the action weights (`eval\_harness.py:116`): higher difficulty means the signal distinguishing optimal from suboptimal actions is weaker. Lower difficulty means clearer signal.

3. ****Smaller state dimension.**** 8-9 base dimensions vs. 10-14 for harder games. Fewer parameters to learn with the same 280-step budget.

4. ****Random baseline concern (revisited).**** For 2-action games, a random agent picks optimally 50% of the time, yielding an expected reward around +0.325. The best results (+0.454, +0.316, +0.296) are in the same ballpark. It is plausible that the positive-game learners are performing only marginally above random chance.

7. Tell Sensitivity Analysis

The sensitivity sweep (tell_scales: 0.0, 0.5, 1.0, 1.5) reveals:

****Positive games (IPD, stag_hunt):**** Performance is positive across all tell scales, including tell_scale=0.0 (tells zeroed out). For IPD A2C, tell_scale=0.0 gives mean=+0.234 and tell_scale=1.0 gives mean=+0.450. The tell features provide some signal but performance is positive even without them, suggesting the agents are mostly learning from base state and profile features, not from tells.

****Negative games (bertrand, poker):**** Performance is negative at all tell scales. For bertrand A2C: tell_scale=0.0 gives mean=-0.354, tell_scale=1.0 gives mean=-0.385. Varying tell intensity does not rescue performance. For poker A2C: tell_scale=0.0 gives mean=-0.287, tell_scale=0.5 gives mean=-0.145, tell_scale=1.0 gives mean=-0.281. The non-monotonic pattern suggests the agents are not learning meaningful tell-to-action mappings.

Overall, tell sensitivity shows that tells are not a primary driver of performance in either direction.

8. Recommendations

Immediate (Low Effort)

1. ****Add random and oracle baselines.**** Compute expected reward for uniform-random and always-optimal policies. Report all results as "reward above random" and "fraction of oracle performance." This single change would dramatically clarify whether the learners are doing anything useful.

2. ****Apply Bonferroni correction**** to reported p-values. Report both raw and adjusted values.

3. ****Increase training budget.**** Even 2,800 steps (10x current) with linear models would provide a fairer test. The current 280 steps is insufficient for any nontrivial learning.

Medium Term (Moderate Effort)

4. ****Add at least one hidden layer**** (e.g., 64- or 128-unit MLP with ReLU). This addresses the

fundamental representational bottleneck. A single hidden layer would allow learning nonlinear decision boundaries.

5. **Implement proper DQN components:** experience replay buffer (even a small one, 1000-5000 transitions) and a target network (update every 50-100 steps). These are essential for stable Q-learning.

6. **Add entropy bonus to A2C.** Standard coefficient of 0.01-0.05 on the entropy of the policy distribution.

7. **Per-game hyperparameter tuning.** At minimum, grid-search over learning rate and exploration parameters per game category (2-action vs. many-action).

Longer Term (Significant Effort)

8. **Replace synthetic environments with real game-theoretic payoff structures.** Use actual payoff matrices for matrix games, actual auction mechanisms for auction games, etc. The current `LightweightGameSimulator` makes all games structurally identical.

9. **Implement multi-agent training.** Self-play or population-based training for games where opponent interaction matters.

10. **Add opponent adaptation.** Dynamic opponents that adjust strategy in response to the agent's behavior, rather than fixed profiles.

11. **Curriculum learning.** Start with low difficulty and ramp up. Start with fewer actions and add complexity.

12. **Proper model selection.** Cross-validation or separate holdout seeds for algorithm selection vs. performance reporting.

9. Summary Verdict

This is a simulation framework, not a competitive game-playing system. The 83% failure rate (15/18 negative) is a direct consequence of fundamental design choices: linear function approximation, minimal training budget, synthetic environments, and stripped-down algorithm implementations. The 3 successes on 2-action/low-difficulty games may not meaningfully exceed a random baseline. The framework demonstrates the mechanics of running multi-method RL experiments but does not yet produce agents that play games effectively.

Missed Games Insights

Missed Games Insights

Aggregate

- Algorithm win counts: PPO=5, A2C=6, DQN=5
- Games with all methods positive: hawk_dove, iterated_prisoners_dilemma, stag_hunt
- Games with all methods negative: auctions, beer_distribution, bertrand, cournot, crisis_negotiations, interrogation, market_for_lemons, principal_agent, public_goods, signaling, tenders, trust_game, ultimatum
- Strongest tell-sensitivity delta (1.5-0.0): ultimatum A2C delta=-0.2188 (-0.4039->-0.6227)

Per Game

| Game | Best | Mean | 95% CI | p(vs 0) | Pairwise p<0.05 |
|----------------------------|------|---------|--------------------|----------|-----------------|
| auctions | PPO | -0.3361 | [-0.4408, -0.2314] | 3.14e-10 | - |
| beer_distribution | DQN | -0.2370 | [-0.3539, -0.1201] | 7.06e-05 | - |
| bertrand | PPO | -0.3720 | [-0.4796, -0.2644] | 1.24e-11 | - |
| cournot | A2C | -0.3435 | [-0.4483, -0.2387] | 1.33e-10 | - |
| crisis_negotiations | A2C | -0.2076 | [-0.3423, -0.0729] | 0.00252 | - |
| hawk_dove | A2C | +0.3265 | [+0.1610, +0.4921] | 0.000111 | PPO_minus_A2C |
| interrogation | A2C | -0.3491 | [-0.4624, -0.2358] | 1.54e-09 | - |
| iterated_prisoners_dilemma | PPO | +0.4419 | [+0.2618, +0.6221] | 1.52e-06 | - |
| market_for_lemons | DQN | -0.2646 | [-0.3871, -0.1421] | 2.29e-05 | - |
| principal_agent | PPO | -0.2390 | [-0.3512, -0.1269] | 2.94e-05 | - |
| public_goods | PPO | -0.3808 | [-0.5070, -0.2545] | 3.4e-09 | - |
| signaling | DQN | -0.1611 | [-0.2836, -0.0387] | 0.00991 | - |
| stag_hunt | DQN | +0.3327 | [+0.2003, +0.4652] | 8.52e-07 | - |
| tenders | DQN | -0.3519 | [-0.4177, -0.2862] | 0 | - |
| trust_game | A2C | -0.3168 | [-0.4363, -0.1973] | 2.04e-07 | - |
| ultimatum | A2C | -0.3550 | [-0.4727, -0.2374] | 3.31e-09 | - |